

Overview of the package LMMstar

Brice Ozenne

October 8, 2026

This vignette describes the main functionalities of the **LMMstar** package.

1 Statistical background

The LMMstar package implements a specific class of linear mixed models, mainly useful when having repeated observations over a discrete variable: $\mathbf{Y} = (Y_1, \dots, Y_T)$ where T can be for example be time (chronological ordering of the repetitions) or brain region (arbitrary ordering of the repetitions). Denoting by $\mathbf{X}$ and $\mathbf{Z}$ two, possibly empty or distinct or overlapping, sets of covariates, the model can be written:

$$\mathbf{Y} = \mathbf{X}\beta + \boldsymbol{\varepsilon} \text{ where } \boldsymbol{\varepsilon} \sim \mathcal{N}(0, \Omega(\mathbf{Z}, \gamma))$$

where $\boldsymbol{\varepsilon} = (\varepsilon_1, \dots, \varepsilon_T)$ are residuals, β mean parameters, γ variance-covariance parameters leading to the modeled residual variance-covariance matrix Ω and. The model parameters $\Theta = (\beta, \gamma)$ can be estimated using Maximum Likelihood (ML) or Restricted Maximum Likelihood (REML) - see appendix B.1. Key assumptions are:

- β and γ are distinct sets of parameters and the residual variance is independent of the mean value.
- we observe n independent replicates of $\mathcal{O} = (\mathbf{Y}, \mathbf{X}, \mathbf{Z})$, i.e. at the cluster level, observations $(\mathcal{O}_1, \dots, \mathcal{O}_n)$ are independent. Given $\mathbf{X}$ and $\mathbf{Z}$ the replicates are identically distributed.

Additional assumptions are necessary in presence of missing values to obtain consistent estimates of the mean parameters:

- correct specification of the conditional mean
- time elapsed between follow-up times are independent of current and previously unobserved outcomes given previously observed outcomes and covariates (Lipsitz et al., 2002).

Fitted mean and variance can be obtained conditionally on covariates only, or also conditionnally on previous outcomes (say $Y_1, \dots, Y_s$), assuming in the latter case a joint Gaussian distribution. For instance:

$$\begin{aligned} \mathbb{E}[Y_t | Y_{1:s}, \mathbf{X}, \mathbf{Z}] &= X_{t,t}\beta + \Omega_{t,1:s}(\gamma, \mathbf{Z})\Omega_{1:s,1:s}^{-1}(\gamma, \mathbf{Z})(Y_{1:s} - X_{1:s}\beta) \\ \text{Var}[Y_t | Y_{1:s}, \mathbf{X}, \mathbf{Z}] &= \Omega_{t,t}(\gamma, \mathbf{Z}) - \Omega_{t,1:s}(\gamma, \mathbf{Z})\Omega_{1:s,1:s}^{-1}(\gamma, \mathbf{Z})\Omega_{1:s,t}(\gamma, \mathbf{Z}) \end{aligned}$$

Multiple imputation of the missing outcomes can then be performed, still under Gaussian distribution assumption.

Wald tests can be used to perform statistical inference on linear combination(s) of model parameters or smooth non-linear combination(s) using a first order delta-method:

- by default 'model-based' standard errors are evaluated, as the square root of the diagonal elements of the inverse of the observed information: $\mathcal{I}(\Theta) = -\mathcal{H}(\Theta)$. Here $\mathcal{H}(\Theta)$ refers to the Hessian, see appendix B.3 for an explicit formula. Using the observed information has been recommended (Thomadakis et al., 2023) over the expected information, $\mathcal{I}(\Theta) = -\mathbb{E}[\mathcal{H}(\Theta)|\mathbf{X}]$, in presence of missing data. The expected information is also implemented to match results of software.
- 'robust' standard error are evaluated as the square root of the diagonal elements of $\mathcal{I}(\Theta)^{-1} \left(\sum_{i=1}^n \mathcal{S}_i(\Theta) \mathcal{S}_i^\top(\Theta) \right) \mathcal{I}(\Theta)^{-1}$.  Inflated type 1 error rate is expect in small samples as no small sample correction is currently implemented.
- degrees of freedom are computed using a Satterthwaite approximation, using numerical derivative of the information.
- adjustment for multiple comparisons can be performed using various methods, including a max-test adjustment either by numerical integration (via the multcomp package) or by random sampling in the join distribution of the test statistics under the null (via the copula package)

The package also supports test on parameters originating from separate linear mixed models, as soon as a common 'cluster' and 'time' variable is used.

- parameters can be pooled, e.g., based on standard errors (meta-analysis), standard error and correlation (Ozenne et al., 2025), using Rubin's rule (multiple imputation).
- statistical inference is performed by combining the influence functions originating from each model (Pipper et al., 2012).

The influence function is computed as $\mathcal{I}^{-1} \mathcal{S}_i(\Theta)$:

- by default it is rescaled by the ratio between the 'robust' and the 'model-based' standard errors so that its sum of square equals the 'model-based' and not the 'robust' standard errors.
- the REML term in $\mathcal{S}_i(\Theta)$ is evaluated as: $tr \left(\left(\sum_{i=1}^n w_i \mathbf{X}_i \Omega_i^{-1}(\Theta) \mathbf{X}_i^\top \right)^{-1} \left(w_i \mathbf{X}_i \Omega_i^{-1}(\Theta) \frac{\partial \Omega_i(\Theta)}{\partial \theta} \Omega_i(\Theta)^{-1} \mathbf{X}_i^\top \right) \right)$ i.e. treating the denominator as a fixed quantity.

2 Summary of the user-interface

To get start, one should load the **LMMstar** package in the  session:

```
library(LMMstar)
```

This package is under active development. Newer package versions may include additional functionalities and fix previous bugs. The version of the package that is being used for this overview is:

```
utils::packageVersion("LMMstar")
```

```
[1] '1.1.0'
```

It is recommended to also load the following packages, as some of the methods implemented in the package are relative to a generic method implemented in other packages:

```
library(ggplot2) ## autoplot method  
library(nlme) ## ranef method  
library(lava) ## iid, information, manifest methods
```

The user interface of the **LMMstar** package is made of the following functions:

- functions to describe or visualize the dataset:
 - `scatterplot` to visualize the marginal and bivariate distribution of continuous variables.
 - `summarize` to compute summary statistics, possibly stratified on a categorical variable.
 - `summarizeNA` to identify missing data patterns.
 - `partialCor` to compute partial correlation between two variables.
- the function `mt.test` to perform multiple Student's t-Tests and adjust the results for multiple testing.
- the function `lmm` is the main function of the package which fits linear mixed models. The user can interact with *lmm* objects using:
 - `anova` to perform Wald tests, i.e. test linear combinations of coefficients ($\hat{\beta}_1 + \hat{\beta}_2 = 0$ or $\hat{\beta}_1 = \hat{\beta}_2 = 0$). The output obtained with different `lmm` can be combined using `rbind`.
 - `coef` to extract the estimated model parameters ($\hat{\beta}$ and possibly $\hat{\gamma}$).
 - `confint` to extract the estimates with their confidence intervals.
 - `effects` to evaluate marginal effects, e.g. $\mathbb{E}[\mathbb{E}[Y|X_1 = 1] - \mathbb{E}[Y|X_1 = 0]]$ when $\mathbf{X} = (X_1, X_2)$.
 - `estimate` to test non-linear combinations of coefficients (Wald test via a first order delta method, e.g. $\hat{\beta}_1/\hat{\beta}_2 = 1$).
 - `fitted` to output the fitted mean ($X\hat{\beta}$) or the conditional mean for observations with missing outcome (e.g. $X\hat{\beta} + \mathbb{E}[\varepsilon_T|\varepsilon_{-T}]$).
 - `iid` to extract the influence function of the estimated parameters (φ), which satisfies $\sqrt{n}(\hat{\beta} - \beta) = \frac{1}{\sqrt{n}} \sum_{i=1}^n \varphi(\mathcal{O}_i) + o_p(1)$
 - `levels` to extract the reference level for the mean structure. (i.e. what (`Intercept`) refers to in presence of categorical covariates).
 - `logLik` to output the log-likelihood of the estimated model.
 - `model.tables` to extract the estimates, standard errors, p-value, and confidence intervals.
 - `plot` to obtain a diagnostic plots, partial residual plots, or a graphical display of the fitted values.
 - `predict` to compute the mean conditional on covariates and possible outcome values.
 - `profile` to display the likelihood or profile likelihood of the model.
 - `resample` to use non-parametric bootstrap or permutation test for statistical inference.
 - `residuals` to extract the observed residuals of the fitted model, possibly normalized ($\hat{\Omega}^{-\frac{1}{2}}\hat{\varepsilon}$).
 - `sigma` to extract the modeled residual variance covariance matrix ($\hat{\Omega}$).
 - `summary` to obtain a summary of the input, model fit, and estimated values.
 - `vcov` to extract the variance-covariance matrix of the mean parameters ($\hat{\Sigma}_{\hat{\beta}}$).
- the `mlmm` function to fit group-specific linear mixed models and gather the estimated coefficients.
- the `sampleRem` function to simulate longitudinal data.
- the `LMMstar.options` function enables the user to display the default values used in the **LMMstar** package. The function can also change the default values to better match the user needs.

3 Illustrative dataset

To illustrate the functionalities of the package, we will use the `gastricbypass` dataset. The long format can be imported using:

```
data(gastricbypassL, package = "LMMstar")
head(gastricbypassL)
```

	id	visit	time	weight	glucagonAUC
1	1	1	-13	127.2	20.690
2	2	1	-13	165.2	49.922
3	3	1	-13	109.7	42.434
4	4	1	-13	146.2	27.517
5	5	1	-13	113.1	29.151
6	6	1	-13	158.8	42.700

See `?gastricbypassL` for a presentation of the dataset. It is convenient to encode the time variable in two formats:

- numeric, e.g. here with the time in week since surgery (`time` variable taking values -13,-1,1,13 - negative times referring to before an intervention and positive times after the intervention).
- factor, e.g. here with the visit index (`visit` variable taking value 1,2,3,4)

To illustrate certain functionalities we will use an (artificial) group variable:

```
gastricbypassL$group <- as.factor(as.numeric(gastricbypassL$id)%2)
```

and dichotomize time as before and after the intervention:

```
gastricbypassL$baseline <- gastricbypassL$time<0
```

The corresponding wide format is

```
data(gastricbypassW, package = "LMMstar")
head(gastricbypassW)
```

	id	weight1	weight2	weight3	weight4	glucagonAUC1	glucagonAUC2	glucagonAUC3	glucagonAUC4
1	1	127.2	120.7	115.5	108.1	20.690	20.535	92.600	43.434
2	2	165.2	153.4	149.2	132.0	49.922	58.513	49.633	35.747
3	3	109.7	101.6	97.7	87.1	42.434	25.770	91.240	83.137
4	4	146.2	142.4	136.7	123.0	27.517	27.552	59.360	21.371
5	5	113.1	105.6	99.9	87.7	29.151	NA	86.859	57.970
6	6	158.8	143.6	134.6	108.7	42.700	31.616	53.408	37.636

for which we can also add the group variable:

```
gastricbypassW$group <- as.numeric(gastricbypassW$id)%2
```

In some cases we will, for comparison, perform complete case analyses with the following dataset:

```
gastricbypassL.NNA <- gastricbypassL[!is.na(gastricbypassL$glucagonAUC),]
```

4 Visualization & descriptive statistics

4.1 Correlation and partial correlations

The `partialCor` function can be used to evaluate group-specific correlations, e.g.:

```
partialCor(weight + glucagonAUC ~ 1, by = "group", data = gastricbypassL)
```

		estimate	se	df	lower	upper	p.value
0:	<code>rho(weight,glucagonAUC)</code>	-0.328	0.143	21.8	-0.587	-0.00886	0.0447
1:	<code>rho(weight,glucagonAUC)</code>	-0.354	0.141	22.5	-0.607	-0.03631	0.0313

This will lead to the same estimate as the `cor.test` function (Pearson correlation):

```
gastricbypassL.0 <- gastricbypassL[gastricbypassL$group==0,]  
rho <- cor.test(gastricbypassL.0$weight, gastricbypassL.0$glucagonAUC)  
c(rho$estimate, p.value = rho$p.value)
```

	cor	p.value
	-0.328481	0.038505

However the p-value may differ, especially in small samples, as `partialCor` uses a different (and probably more crude) small sample approximation for the estimator's distribution. Nevertheless `partialCor` enables to compare correlation coefficients across groups, by specifying the argument `effects`:

```
partialCor(weight + glucagonAUC ~ 1, by = "group", effects = "Dunnett",  
           data = gastricbypassL)
```

		estimate	se	df	lower	upper	p.value
1:	<code>rho(weight,glucagonAUC)</code>	-0.0255	NA	NA	NA	NA	0.899

Partial correlations can be also computed by specifying covariate to adjust for on the right-hand side:

```
partialCor(weight4 + glucagonAUC4 ~ weight1,  
           data = gastricbypassW)
```

		estimate	se	df	lower	upper	p.value
	<code>rho(weight4,glucagonAUC4)</code>	0.112	0.233	9.12	-0.397	0.568	0.645

When the set of covariates is outcome-dependent, a list of formulas can be used instead:

```
partialCor(list(weight1 ~ glucagonAUC1, weight4 ~ glucagonAUC4),  
           data = gastricbypassW)
```

		estimate	se	df	lower	upper	p.value
	<code>rho(weight1,weight4)</code>	0.946	0.0252	26.4	0.861	0.979	5.51e-08

These partial correlations are defined as the residual correlation between the outcomes, i.e. the correlation once the covariate effects have been subtracted from the outcome, and a linear mixed model is used to estimate them.

5 Multiple Student's t-tests

When working with multiple outcomes and having no missing data, mean comparisons between exposure groups can be carried out using Student's t-tests at each timepoint, e.g.:

```
restt <- t.test(weight1 ~ group, data = gastricbypassW)
c(estimate = unname(diff(restt$estimate)), p.value = restt$p.value)
```

```
estimate    p.value
-10.60000    0.25282
```

And so on for the three other timepoints. Moreover results would typically need to be adjusted for multiple comparisons, e.g. when looking for any mean difference. This can be facilitated by

```
## single step max-test adjustment (see help(confint.Wald_lmm) for details)
mt.test(weight1+weight2+weight3+weight4~group, data = gastricbypassW)
```

```
      by parameter estimate      se      df      lower      upper p.value
1 weight1      group    -10.60  8.9717  17.965 -30.968   9.7680  0.31894
2 weight2      group     -9.50  8.3951  17.985 -28.559   9.5590  0.34164
3 weight3      group     -8.92  8.1295  17.959 -27.376   9.5358  0.35891
4 weight4      group     -4.59  7.7607  17.682 -22.209  13.0286  0.66331
```

The method used to adjust confidence intervals and p-values for multiple comparisons can be specified via the `method` argument, e.g.:

```
## no adjustment
mt.test(weight1+weight2+weight3+weight4~group, data = gastricbypassW, method = "none")
```

```
      by parameter estimate      se      df      lower      upper p.value
1 weight1      group    -10.60  8.9717  17.965 -29.452   8.2516  0.25281
2 weight2      group     -9.50  8.3951  17.985 -27.139   8.1386  0.27266
3 weight3      group     -8.92  8.1295  17.959 -26.002   8.1622  0.28703
4 weight4      group     -4.59  7.7607  17.682 -20.916  11.7356  0.56171
```

```
## bonferroni adjustment
mt.test(weight1+weight2+weight3+weight4~group, data = gastricbypassW, method = "bonferroni")
```

```
      by parameter estimate      se      df      lower      upper p.value
1 weight1      group    -10.60  8.9717  17.965 -35.498  14.298      1
2 weight2      group     -9.50  8.3951  17.985 -32.795  13.795      1
3 weight3      group     -8.92  8.1295  17.959 -31.481  13.641      1
4 weight4      group     -4.59  7.7607  17.682 -26.165  16.985      1
```

6 Linear mixed model (LMM)

6.1 Classical covariance patterns

Several build-in covariance patterns can be used when specifying the linear model. The most basic ones are the **identity** structure:

```
eId.lmm <- lmm(glucagonAUC ~ visit*group, repetition = ~time|id,  
              structure = "ID", data = gastricbypassL)  
eId.lmm  
cat(" modeled residual variance-covariance: \n");sigma(eId.lmm)
```

Linear regression

```
outcome/cluster/time: glucagonAUC/id/time  
data                  : 78 observations from 20 clusters  
parameter             : 8 mean ((Intercept) visit2 visit3 visit4 group1 visit2:group1 visit3:group1  
                        1 variance (sigma)  
log-restr.likelihood: -316.461119970244  
convergence           : TRUE (0 iterations)  
modeled residual variance-covariance:  
      -13      -1       1       13  
-13 381.35   0.00   0.00   0.00  
-1   0.00 381.35   0.00   0.00  
1    0.00  0.00 381.35   0.00  
13   0.00  0.00  0.00 381.35
```

and the **independence** structure:

```
eInd.lmm <- lmm(glucagonAUC ~ visit*group, repetition = ~time|id,  
              structure = "IND", data = gastricbypassL)  
eInd.lmm  
cat(" modeled residual variance-covariance: \n");sigma(eInd.lmm)
```

Linear regression with heterogeneous residual variance

```
outcome/cluster/time: glucagonAUC/id/time  
data                  : 78 observations from 20 clusters  
parameter             : 8 mean ((Intercept) visit2 visit3 visit4 group1 visit2:group1 visit3:group1  
                        4 variance (sigma k.-1 k.1 k.13)  
log-restr.likelihood: -310.428096419287  
convergence           : TRUE (0 iterations)  
modeled residual variance-covariance:  
      -13      -1       1       13  
-13 209.44   0.00   0.00   0.00  
-1   0.00 174.81   0.00   0.00  
1    0.00  0.00 768.23   0.00  
13   0.00  0.00  0.00 382.95
```

The most common linear mixed model uses a **compound symmetry** structure:

```
eCS.lmm <- lmm(glucagonAUC ~ visit*group, repetition = ~time|id,
               structure = "CS", data = gastricbypassL)
eCS.lmm
cat(" modeled residual variance-covariance: \n");sigma(eCS.lmm)
```

Linear Mixed Model with a compound symmetry covariance matrix

```
outcome/cluster/time: glucagonAUC/id/time
data                  : 78 observations from 20 clusters
parameter             : 8 mean ((Intercept) visit2 visit3 visit4 group1 visit2:group1 visit3:group1
                        1 variance (sigma)
                        1 correlation (rho(id))
log-restr.likelihood: -314.394203759159
convergence           : TRUE (6 iterations)
modeled residual variance-covariance:
      -13      -1       1       13
-13 380.580  82.741  82.741  82.741
-1   82.741 380.580  82.741  82.741
1    82.741  82.741 380.580  82.741
13   82.741  82.741  82.741 380.580
```

A more flexible model can be obtained with a **toeplitz** covariance matrix:

```
eTOE.lmm <- lmm(glucagonAUC ~ visit*group, repetition = ~time|id,
                structure = "TOEPLITZ", data = gastricbypassL)
eTOE.lmm
cat(" modeled residual correlation: \n");cov2cor(sigma(eTOE.lmm))
```

Linear Mixed Model with a block Toeplitz covariance matrix

```
outcome/cluster/time: glucagonAUC/id/time
data                  : 78 observations from 20 clusters
parameter             : 8 mean ((Intercept) visit2 visit3 visit4 group1 visit2:group1 visit3:group1
                        4 variance (sigma k.-1 k.1 k.13)
                        4 correlation (rho(12) rho(14) rho(26) rho(2))
log-restr.likelihood: -297.525485582536
convergence           : TRUE (15 iterations)
modeled residual correlation:
      -13      -1       1       13
-13 1.000000 0.700020 0.093615 -0.082963
-1   0.700020 1.000000 0.016795  0.093615
1    0.093615 0.016795 1.000000  0.700020
13   -0.082963 0.093615 0.700020  1.000000
```

And an even more flexible model can be obtained with an **unstructured** covariance matrix:

```
eUN.lmm <- lmm(glucagonAUC ~ visit*group, repetition = ~time|id,
               structure = "UN", data = gastricbypassL)
eUN.lmm
cat(" modeled residual variance-covariance: \n");sigma(eUN.lmm)
```

Linear Mixed Model with an unstructured covariance matrix

```
outcome/cluster/time: glucagonAUC/id/time
data                  : 78 observations from 20 clusters
parameter             : 8 mean ((Intercept) visit2 visit3 visit4 group1 visit2:group1 visit3:group1
                        4 variance (sigma k.-1 k.1 k.13)
                        6 correlation (rho(-13,-1) rho(-13,1) rho(-13,13) rho(-1,1) rho(-1,13) rho(1,13))
log-restr.likelihood: -295.314056198772
convergence           : TRUE (8 iterations)
modeled residual variance-covariance:
      -13      -1       1       13
-13 209.442 150.2502 106.4000 -24.202
-1  150.250 168.1138  1.3064 -23.884
1   106.400  1.3064 748.0769 288.184
13  -24.202 -23.8844 288.1839 382.952
```

Stratification of the covariance structure on a categorical variable is also possible:

- e.g. to get a **stratified compound symmetry**

```
eSCS.lmm <- lmm(glucagonAUC ~ visit*group, repetition = ~time|id,
                structure = CS(group~1), data = gastricbypassL)
eSCS.lmm
```

Linear Mixed Model with a stratified compound symmetry covariance matrix

```
outcome/cluster/time: glucagonAUC/id/time
data                  : 78 observations from 20 clusters
parameter             : 8 mean ((Intercept) visit2 visit3 visit4 group1 visit2:group1 visit3:group1 v
                        2 variance (sigma:0 sigma:1)
                        2 correlation (rho(id):0 rho(id):1)
log-restr.likelihood: -314.123797063042
convergence           : TRUE (7 iterations)
```

- e.g. **stratified unstructured** covariance matrix:

```
eSUN.lmm <- lmm(glucagonAUC ~ visit*group, repetition = ~time|id,
               structure = UN(group~1), data = gastricbypassL)
eSUN.lmm
```

Linear Mixed Model with a stratified unstructured covariance matrix

```
outcome/cluster/time: glucagonAUC/id/time
data                  : 78 observations from 20 clusters
parameter             : 8 mean ((Intercept) visit2 visit3 visit4 group1 visit2:group1 visit3:group1 v
                       8 variance (sigma:0 sigma:1 k.-1:0 k.1:0 k.13:0 k.-1:1 k.1:1 k.13:1)
                       12 correlation (rho(-13,-1):0 rho(-13,1):0 rho(-13,13):0 rho(-1,1):0 rho(-1,13):0 rho(1,-1):0 rho(1,1):0 rho(1,13):0)
log-restr.likelihood: -286.536815485471
convergence           : TRUE (10 iterations)
```

with modeled residual variance-covariance:

sigma(eSCS.lmm)	sigma(eSUN.lmm)
<pre>\$'0' -13 -1 1 13 -13 334.289 50.782 50.782 50.782 -1 50.782 334.289 50.782 50.782 1 50.782 50.782 334.289 50.782 13 50.782 50.782 50.782 334.289 \$'1' -13 -1 1 13 -13 428.46 115.09 115.09 115.09 -1 115.09 428.46 115.09 115.09 1 115.09 115.09 428.46 115.09 13 115.09 115.09 115.09 428.46</pre>	<pre>\$'0' -13 -1 1 13 -13 309.85 251.512 102.189 -42.250 -1 251.51 274.752 -79.811 -90.718 1 102.19 -79.811 579.110 163.767 13 -42.25 -90.718 163.767 173.439 \$'1' -13 -1 1 13 -13 109.0309 48.667 104.908 -6.1549 -1 48.6665 59.395 93.976 43.2144 1 104.9077 93.976 967.583 450.8899 13 -6.1549 43.214 450.890 592.4655</pre>

Finally the some covariance patterns like the compound symmetry structure may depend on covariates:

- e.g. to obtain a **block compound symmetry** structure¹:

```
eBCS.lmm <- lmm(glucagonAUC ~ visit*group, repetition = ~time|id,
               structure = CS(~baseline, type = "homogeneous"), data = gastricbypassL)
eBCS.lmm
cat(" modeled residual variance-covariance: \n");sigma(eBCS.lmm)
```

Linear Mixed Model with a block compound symmetry covariance matrix

```
outcome/cluster/time: glucagonAUC/id/time
data                  : 78 observations from 20 clusters
parameter            : 8 mean ((Intercept) visit2 visit3 visit4 group1 visit2:group1 visit3:group1
                      1 variance (sigma)
                      2 correlation (rho(id/baseline) rho(id))
log-restr.likelihood: -308.994835006264
convergence           : TRUE (6 iterations)
modeled residual variance-covariance:
      -13      -1       1       13
-13 380.957 226.403 15.465 15.465
-1 226.403 380.957 15.465 15.465
1 15.465 15.465 380.957 226.403
13 15.465 15.465 226.403 380.957
```

- e.g. to obtain a **block unstructured** covariance matrix:

```
eBUN.lmm <- lmm(glucagonAUC ~ visit*group, repetition = ~time|id,
               structure = CS(~baseline, type = "heterogeneous"), data = gastricbypassL)
eBUN.lmm
cat(" modeled residual variance-covariance: \n");sigma(eBUN.lmm)
```

Linear Mixed Model with a block unstructured covariance matrix

```
outcome/cluster/time: glucagonAUC/id/time
data                  : 78 observations from 20 clusters
parameter            : 8 mean ((Intercept) visit2 visit3 visit4 group1 visit2:group1 visit3:group1
                      2 variance (sigma k.TRUE)
                      3 correlation (rho(FALSE) rho(FALSE,TRUE) rho(TRUE))
log-restr.likelihood: -300.047474124556
convergence           : TRUE (7 iterations)
modeled residual variance-covariance:
      -13      -1       1       13
-13 189.420 150.356 15.353 15.353
-1 150.356 189.420 15.353 15.353
1 15.353 15.353 570.908 300.071
13 15.353 15.353 300.071 570.908
```

¹similar to nested random effects

6.2 User-specific covariance patterns

It is possible input user-specific covariance patterns under the following model for the residuals:

$$\Omega = \sigma^\top R \sigma$$

- $\sigma = f(\theta_\sigma, Z_\sigma)$ is a vector of residual standard errors depending on a vector of parameters θ_σ and possible covariates via the design matrix Z_σ .
- $R = g(\theta_R, Z_R)$ is a matrix of residual correlations depending on a vector of parameters θ_R and possible covariates via the design matrix Z_R .

To be more concrete, consider the following correlation matrix

```
rho.2block <- function(p,n.time,X){
  rho <- matrix(1, nrow = n.time, ncol = n.time)
  rho[1,2] <- rho[2,1] <- rho[4,5] <- rho[5,4] <- p["rho1"]
  rho[1,3] <- rho[3,1] <- rho[4,6] <- rho[6,4] <- p["rho2"]
  rho[2,3] <- rho[3,2] <- rho[5,6] <- rho[6,5] <- p["rho3"]
  rho[4:6,1:3] <- rho[1:3,4:6] <- p["rho4"]
  return(rho)
}
Rho <- rho.2block(p = c(rho1=0.25,rho2=0.5,rho3=0.4,rho4=0.1),
  n.time = 6)
Rho
```

```
      [,1] [,2] [,3] [,4] [,5] [,6]
[1,] 1.00 0.25  0.5 0.10 0.10  0.1
[2,] 0.25 1.00  0.4 0.10 0.10  0.1
[3,] 0.50 0.40  1.0 0.10 0.10  0.1
[4,] 0.10 0.10  0.1 1.00 0.25  0.5
[5,] 0.10 0.10  0.1 0.25 1.00  0.4
[6,] 0.10 0.10  0.1 0.50 0.40  1.0
```

and the corresponding dataset:

```
set.seed(11)
Y <- mvtnorm::rmvnorm(1000, mean = rep(0,6), sigma = Rho)
dfW <- cbind(id = 1:NROW(Y), as.data.frame(Y))
dfL <- reshape(dfW, direction = "long", idvar = "id",
  timevar = "time", times = paste0("V",1:6),
  varying = paste0("V",1:6), v.names = "value")
dfL[dfL$id %in% 1:2,]
```

	id	time	value		id	time	value	
	1.V1	1	V1	-0.98421	2.V1	2	V1	1.24027
	1.V2	1	V2	-0.36812	2.V2	2	V2	0.64942
	1.V3	1	V3	-1.61747	2.V3	2	V3	0.32721
	1.V4	1	V4	-1.49941	2.V4	2	V4	-1.06270
	1.V5	1	V5	0.74931	2.V5	2	V5	-0.90132
	1.V6	1	V6	-1.07197	2.V6	2	V6	-0.66967

To estimate the corresponding mixed model we first define a new covariance structure:

```
myStruct <- CUSTOM(~time,
  FCT.sigma = function(p,n.time,X){rep(p,n.time)}, ## function f
  init.sigma = c("sigma"=1),
  FCT.rho = rho.2block, ## function g
  init.rho = c("rho1"=0.25,"rho2"=0.25,"rho3"=0.25,"rho4"=0.25))
```

and then call `lmm` with this structure structure:

```
e.lmmCUSTOM <- lmm(value~time, repetition=~time|id,
  structure = myStruct, data=dfL,
  df = FALSE) ## df = FALSE to save computation time
logLik(e.lmmCUSTOM)
```

```
[1] -7962.243
```

The optimization procedure may be slow but should eventually reaches an optimum. We can then output the estimated correlation matrix:

```
cov2cor(sigma(e.lmmCUSTOM))
```

	V1	V2	V3	V4	V5	V6
V1	1.00000000	0.24898095	0.50058994	0.09053785	0.09053785	0.09053785
V2	0.24898095	1.00000000	0.36110943	0.09053785	0.09053785	0.09053785
V3	0.50058994	0.36110943	1.00000000	0.09053785	0.09053785	0.09053785
V4	0.09053785	0.09053785	0.09053785	1.00000000	0.24898095	0.50058994
V5	0.09053785	0.09053785	0.09053785	0.24898095	1.00000000	0.36110943
V6	0.09053785	0.09053785	0.09053785	0.50058994	0.36110943	1.00000000

Comparison to build-in structure: consider the following model using a build-in compound symmetry structure:

```
system.time(
  e.lmmDEFAULT.CS <- lmm(value~time, repetition = ~time|id,
    structure = "CS", data = dfL,
    df = FALSE)
)
```

```
user system elapsed
0.097  0.000  0.097
```

Using instead `CUSTOM` to specifying this structure:

```
myCS <- CUSTOM(~1,
  FCT.sigma = function(p,n.time,X){rep(p,n.time)},
  init.sigma = c("sigma"=1),
  FCT.rho = function(p,n.time,X){p+diag(1-p,n.time,n.time)},
  init.rho = c("rho"=0.5))
```

is considerably slower than using the pre-specified structure:

```
system.time(  
  e.lmmCUSTOM.CS <- lmm(value~time, repetition = ~time|id,  
                        structure = myCS, data = dfL,  
                        df = FALSE)  
)
```

```
user system elapsed  
0.952  0.019  0.972
```

but will lead to the same estimates:

```
logLik(e.lmmDEFAULT.CS)  
logLik(e.lmmCUSTOM.CS)
```

```
[1] -8186.859  
[1] -8186.859
```

There are two reasons for the slower execution time: slower evaluation of the derivatives (since they are obtained by numerical differentiation) and worse starting point, as reflected by the larger number of iterations needed to reach convergence:

```
e.lmmDEFAULT.CS$opt$n.iter  
e.lmmCUSTOM.CS$opt$n.iter
```

```
[1] 1  
[1] 4
```

Faster execution time can be obtained by specifying the first and second derivative regarding each parameter:

```
myCS.wD <- CUSTOM(~1,  
  FCT.sigma = function(p,n.time,X){rep(p,n.time)},  
  dFCT.sigma = function(p,n.time,X){list(sigma = rep(1,n.time))},  
  d2FCT.sigma = function(p,n.time,X){list(sigma = rep(0,n.time))},  
  init.sigma = c("sigma"=1),  
  FCT.rho = function(p,n.time,X){p+diag(1-p,n.time,n.time)},  
  dFCT.rho = function(p,n.time,X){list(rho = 1-diag(1,n.time,n.time))},  
  d2FCT.rho = function(p,n.time,X){list(rho = matrix(0,n.time,n.time))},  
  init.rho = c("rho"=0.5))
```

```
system.time(  
  e.lmmCUSTOMwD.CS <- lmm(value~time,  
                          repetition = ~time|id,  
                          structure = myCS.wD,  
                          data = dfL, df = FALSE  
                          )  
)
```

```
user system elapsed  
0.699  0.004  0.703
```

6.3 Estimation procedure

Initialiation: by default the mean parameters are initialized using Ordinary Least Squares (OLS) and the variance and correlation parameters are initialized by minimizing the difference between the observed and residuals variance-covariance matrix. These values can be visualized by specifying the argument `control`:

```
eCS.lmm.bis <- update(eCS.lmm, control = list(trace = 2))
```

Initialization:

(Intercept)	visit2	visit3	visit4	group1	visit2:group1	visit3:group1
38.72897	-4.73433	31.43303	4.52138	-12.82462	3.75946	27.00150
visit4:group1	sigma	rho(id)				
30.22391	19.52828	0.22819				

Loop:

(Intercept)	visit2	visit3	visit4	group1	visit2:group1	visit3:group1
38.72897	-4.73433	31.43303	4.52138	-12.82462	3.80337	27.48103
visit4:group1	sigma	rho(id)				
30.22391	19.50846	0.21741				

Convergence after 6 iterations: max score=1.2413e-05 | max change in coefficient=4.5167e-06

It is possible to input user-defined value:

- for all parameters (vector)

```
init.all <- coef(eCS.lmm, effects = "all")
eCS.lmm.bis <- update(eCS.lmm, control = list(init = init.all, trace = 1))
```

Convergence after 0 iteration: max score=1.2413e-05

- the mean parameters only (vector)

```
init.mean <- coef(eCS.lmm, effects = "mean")
eCS.lmm.bis <- update(eCS.lmm, control = list(init = init.mean, trace = 2))
```

Initialization:

(Intercept)	visit2	visit3	visit4	group1	visit2:group1	visit3:group1
38.72897	-4.73433	31.43303	4.52138	-12.82462	3.80337	27.48103
visit4:group1	sigma	rho(id)				
30.22391	19.52904	0.22849				

Loop:

(Intercept)	visit2	visit3	visit4	group1	visit2:group1	visit3:group1
38.72897	-4.73433	31.43303	4.52138	-12.82462	3.80337	27.48103
visit4:group1	sigma	rho(id)				
30.22391	19.50846	0.21741				

Convergence after 6 iterations: max score=1.4893e-05 | max change in coefficient=5.3866e-06

- a full data variance-covariance matrix (matrix).

```
init.vcov <- sigma(eCS.lmm)
eCS.lmm.bis <- update(eCS.lmm, control = list(init = init.vcov, trace = 1))
```

Convergence after 0 iteration: max score=1.2413e-05

Optimizer: by default the optimizer is a Newton Raphson algorithm with backtracking. At each iteration:

- it computes the first two moments (score, information) according to the current parameters values.
- it updates the variance-covariance parameters according to the gradient multiplied by the inverse of the information.
- it updates the mean parameters by generalized least squares (using the updated variance-covariance parameters).
- it checks whether the log-likelihood at the updated estimates is well defined and higher than at the previous estimates. If this is not the case, the step is re-run with half the update of the variance-covariance parameters (backtracking).

One can modify the maximum number of iterations (**n.iter**), maximum number of backtracking steps (**n.backtracking**), the maximum score (absolute) value over all parameters (**tol.score**) and (absolute) maximum difference in parameter value between two iterations (**tol.param**) used to declare convergence. It is also possible to use another optimizer (**optimizer**). All these elements should be passed to the argument **control** of **lmm** using a list.

6.4 Model output

The `summary` method can be used to display the main information relative to the model fit:

```
summary(eUN.lmm)
```

Linear Mixed Model

Dataset: `gastricbypassL`

- 20 clusters
- 78 observations were analyzed, 2 were excluded because of missing values
- between 3 and 4 observations per cluster

Summary of the outcome and covariates:

```
$ glucagonAUC: num  20.7 49.9 42.4 27.5 29.2 ...
$ visit       : Factor w/ 4 levels "1","2","3","4": 1 1 1 1 1 1 1 1 1 1 ...
$ group       : Factor w/ 2 levels "0","1": 2 1 2 1 2 1 2 1 2 1 ...
reference level: visit=1;group=0
```

Estimation procedure

- Restricted Maximum Likelihood (REML)
- log-likelihood : -295.31
- parameters: mean = 8, variance = 4, correlation = 6
- convergence: TRUE (8 iterations)
- largest `|score|` = 4.6771e-05 for `rho(-1,1)`
|change| = 1.68033723859651e-05 for `visit3:group1`

Residual variance-covariance: unstructured

- correlation structure: `~0 + time`

	-13	-1	1	13
-13	1.0000	0.80072	0.26880	-0.0855
-1	0.8007	1.00000	0.00368	-0.0941
1	0.2688	0.00368	1.00000	0.5384
13	-0.0855	-0.09413	0.53842	1.0000
- variance structure: `~time`

	standard.deviation	ratio
sigma.-13	14.5	1.000
sigma.-1	13.0	0.896
sigma.1	27.4	1.890
sigma.13	19.6	1.352

Fixed effects: glucagonAUC ~ visit * group

	estimate	se	df	lower	upper	p.value	
(Intercept)	38.729	4.576	18	29.114	48.344	< 1e-04	***
visit2	-4.734	2.776	17.5	-10.577	1.109	0.10574	
visit3	31.433	8.63	17.6	13.272	49.594	0.00192	**
visit4	4.521	8.005	18	-12.297	21.34	0.57917	
group1	-12.825	6.472	18	-26.422	0.773	0.06302	.
visit2:group1	3.987	3.996	17.9	-4.41	12.383	0.33169	
visit3:group1	27.571	12.42	17.8	1.461	53.682	0.03963	*
visit4:group1	30.224	11.321	18	6.439	54.008	0.01562	*

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1.

Columns lower and upper contain 95% pointwise confidence intervals for each coefficient.

Model-based standard errors are derived from the observed information (column se).

Degrees of freedom were computed using a Satterthwaite approximation (column df).

Note: the calculation of the degrees of freedom, especially when using the observed information can be quite slow. Setting the arguments `df` to `FALSE` and `type.information` to `"expected"` when calling `lmm` should lead to a more reasonable computation time.

6.5 Extract estimated coefficients

The value of the estimated coefficients can be output using `coef`:

```
coef(eUN.lmm)
```

(Intercept)	visit2	visit3	visit4	group1	visit2:group1	visit3:group1
38.7290	-4.7343	31.4330	4.5214	-12.8246	3.9866	27.5714
visit4:group1						
30.2239						

Variance coefficients can be output by specifying the `effects` argument:

```
coef(eUN.lmm, effects = "variance")
```

sigma	k.-1	k.1	k.13
14.47212	0.89592	1.88991	1.35220

The first coefficient is the residual standard deviation at the reference timepoint (here -13 week) and the remaining coefficient the residual standard deviation at later timepoints relative to the reference timepoint. It is possible to apply specific transformation on the variance coefficients, for instance to obtain the residual variance at each timepoint:

```
coef(eUN.lmm, effects = "variance", transform.k = "sd")
```

sigma.-13	sigma.-1	sigma.1	sigma.13
14.472	12.966	27.351	19.569

6.6 Extract estimated coefficient and associated uncertainty

The uncertainty about the mean coefficients can be obtained using the `model.tables` method ²:

```
model.tables(eUN.lmm)
```

	estimate	se	df	lower	upper	p.value
(Intercept)	38.7290	4.5765	18.003	29.1143	48.34369	1.0891e-07
visit2	-4.7343	2.7759	17.543	-10.5772	1.10851	1.0574e-01
visit3	31.4330	8.6297	17.585	13.2719	49.59411	1.9229e-03
visit4	4.5214	8.0050	17.995	-12.2968	21.33958	5.7917e-01
group1	-12.8246	6.4721	18.003	-26.4219	0.77265	6.3015e-02
visit2:group1	3.9866	3.9957	17.937	-4.4102	12.38329	3.3169e-01
visit3:group1	27.5714	12.4199	17.831	1.4605	53.68232	3.9634e-02
visit4:group1	30.2239	11.3208	17.995	6.4394	54.00840	1.5624e-02

Values for the all correlation parameters can be displayed too, by specifying `effect=c("variance","correlation")`

```
model.tables(eUN.lmm, effect = c("variance","correlation"))
```

	estimate	se	df	lower	upper	p.value
sigma	14.4721183	2.412020	15.3158	10.15148	20.63170	NA
k.-1	0.8959206	0.127032	20.2671	0.66670	1.20396	0.44721963
k.1	1.8899095	0.431098	25.9157	1.18244	3.02067	0.00974152
k.13	1.3521979	0.317550	29.8074	0.83694	2.18468	0.20874407
rho(-13,-1)	0.8007214	0.085177	13.4142	0.52949	0.92343	0.00042923
rho(-13,1)	0.2688043	0.219200	7.9286	-0.26374	0.67576	0.27735748
rho(-13,13)	-0.0854578	0.233981	8.5882	-0.55306	0.42309	0.72505145
rho(-1,1)	0.0036838	0.237237	8.1487	-0.49424	0.49979	0.98798445
rho(-1,13)	-0.0941328	0.233649	8.9191	-0.55697	0.41331	0.69821381
rho(1,13)	0.5384239	0.176221	10.2233	0.05058	0.81883	0.03522642

Because these parameters are constrained (e.g. strictly positive), they uncertainty is by default computed after transformation (e.g. log) and then backtransformed. The column argument can be used to extract more or less information, e.g.:

```
model.tables(eUN.lmm, columns = c("estimate","p.value"))
```

	estimate	p.value
(Intercept)	38.7290	1.0891e-07
visit2	-4.7343	1.0574e-01
visit3	31.4330	1.9229e-03
visit4	4.5214	5.7917e-01
group1	-12.8246	6.3015e-02
visit2:group1	3.9866	3.3169e-01
visit3:group1	27.5714	3.9634e-02
visit4:group1	30.2239	1.5624e-02

²it is equivalent to `confint` method except that by default it also outputs `se` and `p.value`

All parameters can be displayed by specifying `effect="all"`. The functions `add` (resp. `remove`) can be used to add (resp. remove) one or several columns from the default display, e.g.:

```
model.tables(eUN.lmm, columns = add("statistic"))
```

	estimate	se	statistic	df	lower	upper	p.value
(Intercept)	38.7290	4.5765	8.46260	18.003	29.1143	48.34369	1.0891e-07
visit2	-4.7343	2.7759	-1.70552	17.543	-10.5772	1.10851	1.0574e-01
visit3	31.4330	8.6297	3.64242	17.585	13.2719	49.59411	1.9229e-03
visit4	4.5214	8.0050	0.56482	17.995	-12.2968	21.33958	5.7917e-01
group1	-12.8246	6.4721	-1.98151	18.003	-26.4219	0.77265	6.3015e-02
visit2:group1	3.9866	3.9957	0.99772	17.937	-4.4102	12.38329	3.3169e-01
visit3:group1	27.5714	12.4199	2.21995	17.831	1.4605	53.68232	3.9634e-02
visit4:group1	30.2239	11.3208	2.66977	17.995	6.4394	54.00840	1.5624e-02

6.7 Extract estimated residual variance-covariance structure

The method `sigma` can be used to output the modeled residual covariance structure and then converted to a correlation matrix using `cov2cor`:

```
Sigma <- sigma(eUN.lmm)
Sigma
```

```
cov2cor(Sigma)
```

```

      -13      -1       1       13
-13 209.442 150.250 106.400 -24.202
-1  150.250 168.114   1.306 -23.884
1   106.400   1.306 748.077 288.184
13  -24.202 -23.884 288.184 382.952

```

```

      -13      -1       1       13
-13 1.000 0.801 0.269 -0.085
-1  0.801 1.000 0.004 -0.094
1   0.269 0.004 1.000 0.538
13 -0.085 -0.094 0.538 1.000

```

The method can also be used to extract the residual covariance relative to a "known" individual:

```
sigma(eUN.lmm, cluster = 5)
```

```

      -13       1       13
-13 209.442 106.40 -24.202
1   106.400 748.08 288.184
13  -24.202 288.18 382.952

```

or for a new individual:

```
newdata <- data.frame(id = "X", time = c("-13", "-1", "1", "13"))
sigma(eUN.lmm, cluster = newdata)
```

```

      -13      -1       1       13
-13 209.442 150.2502 106.4000 -24.202
-1  150.250 168.1138   1.3064 -23.884
1   106.400   1.3064 748.0769 288.184
13  -24.202 -23.8844 288.1839 382.952

```

6.8 Marginal effects

The `effects` method can be used to evaluate marginal means with respect to a categorical variable:

- $\mathbb{E}[Y_t \mid \text{group}]$

```
effects(eUN.lmm, variable = "group")
```

Average counterfactual outcome
w.r.t 'group' values

	estimate	se	df	lower	upper
group=0(t=-13)	38.729	4.576	18	29.114	48.344
group=0(t=-1)	33.995	4.1	17.9	25.377	42.612
group=0(t=1)	70.162	8.649	17.7	51.968	88.356
group=0(t=13)	43.25	6.188	18	30.249	56.251
group=1(t=-13)	25.904	4.576	18	16.29	35.519
group=1(t=-1)	25.157	4.167	18.7	16.425	33.889
group=1(t=1)	84.909	8.951	18.2	66.115	103.702
group=1(t=13)	60.65	6.188	18	47.649	73.651

- $\mathbb{E}[Y_t - Y_0 \mid \text{group}]$

```
effects(eUN.lmm, type = "change", variable = "group")
```

Average counterfactual change in outcome
w.r.t 'group' values

	estimate	se	df	lower	upper
group=0(dt=-1)	-4.734	2.776	17.5	-10.577	1.109
group=0(dt=1)	31.433	8.63	17.6	13.272	49.594
group=0(dt=13)	4.521	8.005	18	-12.297	21.34
group=1(dt=-1)	-0.748	2.874	18.3	-6.779	5.283
group=1(dt=1)	59.004	8.932	18	40.242	77.767
group=1(dt=13)	34.745	8.005	18	17.927	51.563

- $\mathbb{E}\left[\int_0^T Y_t dt \mid \text{group}\right]$

```
effects(eUN.lmm, type = "auc", variable = "group")
```

Average counterfactual area under the outcome curve
w.r.t 'group' values

	estimate	se	df	lower	upper
group=0(auc)	1220.972	104.098	17.8	1002.072	1439.873
group=1(auc)	1289.782	105.512	18.5	1068.508	1511.056

It can also be used to contrast these marginal means:

- $\mathbb{E}[Y_t \mid \text{group} = 1] - \mathbb{E}[Y_t \mid \text{group} = 0]$

```
effects(eUN.lmm, type = "difference", variable = "group")
```

Difference in average counterfactual outcome
w.r.t 'group' values

	estimate	se	df	lower	upper	p.value
group=1-0(t=-13)	-12.825	6.472	18	-26.422	0.773	0.0630 .
group=1-0(t=-1)	-8.838	5.846	18.3	-21.106	3.43	0.1477
group=1-0(t=1)	14.747	12.447	17.9	-11.409	40.903	0.2516
group=1-0(t=13)	17.399	8.752	18	-0.987	35.785	0.0622 .

- $\mathbb{E}[Y_t - Y_0 \mid \text{group} = 1] - \mathbb{E}[Y_t - Y_0 \mid \text{group} = 0]$

```
effects(eUN.lmm, type = c("change","difference"), variable = "group")
```

Difference in average counterfactual change in outcome
w.r.t 'group' values

	estimate	se	df	lower	upper	p.value
group=1-0(dt=-1)	3.987	3.996	17.9	-4.41	12.383	0.3317
group=1-0(dt=1)	27.571	12.42	17.8	1.461	53.682	0.0396 *
group=1-0(dt=13)	30.224	11.321	18	6.439	54.008	0.0156 *

- $\mathbb{E} \left[\int_0^T Y_t dt \mid \text{group} = 1 \right] - \mathbb{E} \left[\int_0^T Y_t dt \mid \text{group} = 0 \right]$

```
effects(eUN.lmm, type = c("auc","difference"), variable = "group")
```

Difference in average counterfactual area under the outcome curve
w.r.t 'group' values

	estimate	se	df	lower	upper	p.value
group=1-0(auc)	68.809	148.22	18.1	-242.44	380.059	0.648

It is possible to control the set of covariates used to condition on via the `conditional` argument. This can be useful when considering an interaction with a biomarker to obtain biomarker-specific effects.

6.9 Random effects

Mixed model having a compound symmetry structure with positive correlation parameters may be equivalent to random intercept models, possibly with nested random effects. Indeed in some case the residual variance-covariance matrix can then be decomposed as:

$$\Omega = Z\Psi Z^T + \Delta$$

- Z is the design matrix associated to the random effect (e.g. patient id)
- Ψ is the variance-covariance of the random effects
- Δ the residual variance covariance conditional to the random effects.

One can use the `lme4` syntax to fit random intercept models with `lmm`:

```
eRI.lmm <- lmm(glucagonAUC ~ visit*group + (1|id), data = gastricbypassL)
eRI.lmm
```

Linear Mixed Model with a random intercept

```
outcome/cluster/time: glucagonAUC/id/XXtimeXX
data                  : 78 observations from 20 clusters
parameter             : 8 mean ((Intercept) visit2 visit3 visit4 group1 visit2:group1 visit3:group1 v
                        1 variance (sigma)
                        1 correlation (rho(id))
log-restr.likelihood: -314.394203759159
convergence           : TRUE (6 iterations)
```

It is also possible to specify cross or nested random effects, e.g.:

```
eNRI.lmm <- lmm(glucagonAUC ~ visit*group + (1|id/baseline), data = gastricbypassL)
eNRI.lmm
```

Linear Mixed Model with nested random intercepts

```
outcome/cluster/time: glucagonAUC/id/XXtimeXX
data                  : 78 observations from 20 clusters
parameter             : 8 mean ((Intercept) visit2 visit3 visit4 group1 visit2:group1 visit3:group1 v
                        1 variance (sigma)
                        2 correlation (rho(id/baseline) rho(id))
log-restr.likelihood: -308.994835006264
convergence           : TRUE (6 iterations)
```

We obtain the same log-likelihood as, respectively, `eCS.lmm` and `eBCS.lmm`. Indeed, as previously mentioned, with positive residual correlation the random effect structure is equivalent to a compound symmetry structure.

⚠ random slopes are not currently supported in `LMMstar`.

⚠ the proposed implementation can be very inefficient compared to `lme4`.

The joint distribution between the outcome $\mathbf{Y}$ and the random effects $\boldsymbol{\eta}$ can be expressed as:

$$\begin{bmatrix} \mathbf{Y} \\ \boldsymbol{\eta} \end{bmatrix} \sim \mathcal{N} \left(\begin{bmatrix} \boldsymbol{\mu} \\ \mathbf{0} \end{bmatrix}, \begin{bmatrix} \Omega & Z\Psi \\ \Psi Z^\top & \Psi \end{bmatrix} \right)$$

Denote by $\varepsilon_i = \mathbf{Y}_i - \boldsymbol{\mu}_i$ the vector of marginal residuals relative to individual i , Ω_i its variance-covariance matrix, and $\psi_j = (\Psi)_{jj}$ the variance of the j -th random effect. We can re-express the expected value of the j -th random effect for individual i as:

$$\eta_{ij} = \psi_j Z_{ij} \Omega_i^{-1} \varepsilon_i$$

This is what the `ranef` method returns:

<code>head(ranef(eRI.lmm, format = "wide"))</code>	<code>head(ranef(eNRI.lmm, format = "wide"))</code>
--	---

id estimate	id estimate	estimate.FALSE	estimate.TRUE
1 1 -2.51154	1 1 -0.494271	-3.50959	-3.23209
2 2 1.01043	2 2 0.186051	-10.39431	12.93198
3 3 6.08384	3 3 1.088409	9.36327	5.48225
4 4 -6.62350	4 4 -1.219596	-11.06703	-5.56784
5 5 0.39519	5 5 0.081686	-0.71254	1.82672
6 6 -2.73384	6 6 -0.503386	-7.81700	0.95098

It is also possible to extract the variance decomposition by setting the argument `effects` to `"variance"`:

<code>ranef(eRI.lmm, effects = "variance", format = "wide")</code>	<code>ranef(eNRI.lmm, effects = "variance", format = "wide")</code>
--	---

type absolute relative	type absolute relative
1 total 380.580 1.00000	1 total 380.957 1.000000
2 id 82.741 0.21741	2 id 15.465 0.040595
3 residual 297.839 0.78259	3 baseline 210.938 0.553705
	4 residual 154.554 0.405700

Confidence intervals can also be obtained setting the argument `se` to `TRUE` and `format` equal to `"long"`:

<code>head(ranef(eRI.lmm, se = TRUE))</code>
--

id estimate	se	df	lower	upper
1 1 -2.51154	2.3019	11.1302	-7.5708	2.5477
2 2 1.01043	2.1163	15.7355	-3.4821	5.5030
3 3 6.08384	2.9771	6.2085	-1.1421	13.3098
4 4 -6.62350	3.1114	5.8319	-14.2902	1.0432
5 5 0.39519	1.9661	23.8446	-3.6640	4.4543
6 6 -2.73384	2.2940	10.0189	-7.8438	2.3761

6.10 Sum of squares

⚠ The definition of the sum of squares is not straightforward with mixed models. Intuitively summing residuals across several outcomes will be hard to interpret unless all outcomes have the same variance. This is why LMMstar does not provide them. Nevertheless for specific covariance structure, namely independence and compound symmetry (with positive correlation) structure, sum of squares can be deduced from the `lmm` object - see appendix D for the theoretical derivations. Importantly, with these structures the residuals can be reparametrised as random effects plus independent residuals, i.e. $\Omega = Z\Psi Z^\top + \delta I$ where I is the identity matrix and δ the variance of these independent residuals.

Appendix D illustrate how to extract the sum of squares for univariate linear regression (i.e. independence structure) and here we illustrate the case of a compound symmetry structure. A key step is to extract from the `lmm` object the conditional residual variance δ :

```
sigma2 <- coef(eCS.lmm, effect = "variance")^2
tau <- coef(eCS.lmm, effect = "correlation")*sigma2
delta <- unname(sigma2 - tau)
```

This step will typically depend on the covariance structure. The residual sum of squares (SSE) equals the residual degrees of freedom times the conditional variance:

```
df.res <- df.residual(eCS.lmm)
SSE <- df.res * delta
c(df.res = df.res, SSE = SSE)
```

```
df.res    SSE
    70 20849
```

For the regression sum of squares (SSR), we first extract the mean parameters and their variance-covariance based on the expected information:

```
eBeta.lmm <- coef(eCS.lmm)
eVcov.lmm <- vcov(eCS.lmm, type.information = "expected")
```

Parameters are grouped with respect to the original variable:

```
attr(model.matrix(eCS.lmm), "assign")
```

```
[1] 0 1 1 1 2 3 3 3
```

So we respect this grouping when computing the normalized SSR:

```
SSRstar.time <- eBeta.lmm[2:4] %*% solve(eVcov.lmm[2:4,2:4]) %*% eBeta.lmm[2:4]
SSRstar.group <- eBeta.lmm[5] %*% solve(eVcov.lmm[5,5]) %*% eBeta.lmm[5]
```

The SSR is obtained by multiplying the normalized SSR by the conditional variance:

```
SSR.time <- as.double(SSRstar.time * delta)
SSR.group <- as.double(SSRstar.group * delta)
c(time = SSR.time, group = SSR.group)
```

```
time    group
7872.19 643.57
```

6.11 Proportion of explained variance and partial correlation

For a univariate linear model with homoschedastic residual variance, the proportion of explained variance, also called partial R^2 or partial η^2 , is defined as the ratio between sum of squares (e.g. [Lakens \(2013\)](#), equation 12):

$$R^2 = \frac{SSR}{SSR + SSE}$$

```
c(SSR.time/ (SSR.time + SSE),
  SSR.group/ (SSR.group + SSE))
```

```
[1] 0.274092 0.029944
```

Computing the SSR for each individual coefficients, taking its squared root, and multiplying by the sign of the corresponding coefficient leads to the partial correlation. This procedure extends to covariance structures that can be reparametrised as random effects plus independent residuals (see previous subsection) such as the compound symmetry with non-negative correlation.

⚠ for other covariance structures, especially when the variance may be repetition-dependent, the definition of explained variance/partial correlation is not straightforward.

```
eCS.R2 <- partialCor(eCS.lmm, R2 = TRUE)
summary(eCS.R2)
```

Partial correlation

	estimate	se	df	lower	upper	p.value
visit2	-0.073	0.119	52.4	-0.311	0.165	0.54028
visit3	0.438	0.089	51.4	0.26	0.616	< 1e-04
visit4	0.07	0.119	52.4	-0.168	0.308	0.55876
group1	-0.173	0.114	60.7	-0.402	0.056	0.13527
visit2:group1	0.041	0.119	52.8	-0.198	0.28	0.73256
visit3:group1	0.284	0.106	52	0.071	0.497	0.01007
visit4:group1	0.314	0.103	52	0.107	0.521	0.00365

Columns lower and upper contain 95% pointwise confidence intervals for each coefficient. Degrees of freedom were computed using a Satterthwaite approximation (column df).

Coefficient of determination (R2)

	estimate	se	df	lower	upper	p.value
visit	0.274	0.08	50.5	0.114	0.434	0.0012
group	0.03	0.04	60.7	-0.049	0.109	0.4520
visit:group	0.147	0.073	51.7	<0.001	0.295	0.0500
global	0.598	0.053	40.4	0.492	0.705	<1e-04

Columns lower and upper contain 95% pointwise confidence intervals for each coefficient. Degrees of freedom were computed using a Satterthwaite approximation (column df).

Here the line "global" refer to the R2 for all covariates, computed based on the SSR relative to all mean parameters but the intercept.

⚠ `partialCor` will compute values for all types of mixed models. But their interpretation as partial correlation and proportion of explained variance outside the compound symmetry with non-negative correlation is questionable.

Note: Other software packages like `effectsize::eta_squared` uses another formula to estimate the partial R2:

$$R^2 = \frac{F df_{num}}{F df_{num} + df_{denom}}$$

where F denote the F-statistic, df_{num} (resp. df_{denom}) the degrees of freedom of the numerator (resp. denominator) of this statistic. However since the calculation of degrees of freedom in LMM is approximate, I would expect this approach to be less reliable than the one of `partialCor` based on the SSR and SSE.

```
aCS.aov <- anova(eCS.lmm)$multivariate
setNames(with(aCS.aov, statistic*df.num/(statistic*df.num+df.denom)), aCS.aov$test)
```

visit	group	visit:group
0.335374	0.033811	0.186290

6.12 Model diagnostic

The method `residuals` returns the residuals in the wide format:

```
eUN.diagW <- residuals(eUN.lmm, type = "normalized", format = "wide")
colnames(eUN.diagW) <- gsub("normalized.", "", colnames(eUN.diagW))
head(eUN.diagW)
```

	id	r.-13	r.-1	r.1	r.13
1	1	-0.36029	-0.11344	0.377177	-1.45539
2	2	0.77339	2.12301	-0.232908	-0.10708
3	3	1.14219	-1.44778	-0.654876	2.01259
4	4	-0.77473	0.20612	-0.127117	-1.39519
5	5	0.22435	NA	0.011432	-0.15398
6	6	0.27439	-0.67308	-1.031131	0.42724

or in the long format:

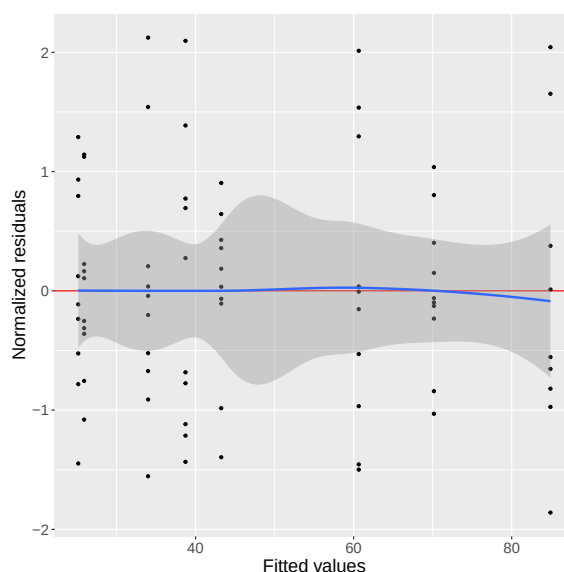
```
eUN.diagL <- residuals(eUN.lmm, type = "normalized", format = "long", keep.data = TRUE)
head(eUN.diagL)
```

	id	visit	time	weight	glucagonAUC	group	baseline	fitted	r.normalized
1	1	1	-13	127.2	20.690	1	TRUE	25.904	-0.36029
2	2	1	-13	165.2	49.922	0	TRUE	38.729	0.77339
3	3	1	-13	109.7	42.434	1	TRUE	25.904	1.14219
4	4	1	-13	146.2	27.517	0	TRUE	38.729	-0.77473
5	5	1	-13	113.1	29.151	1	TRUE	25.904	0.22435
6	6	1	-13	158.8	42.700	0	TRUE	38.729	0.27439

Various type of residuals can be extract but the normalized one are recommended when doing model checking. Diagnostic plots can then be generated by the user, or directly from the `lmm` object via the method `plot` (which internally calls the `residuals` method):

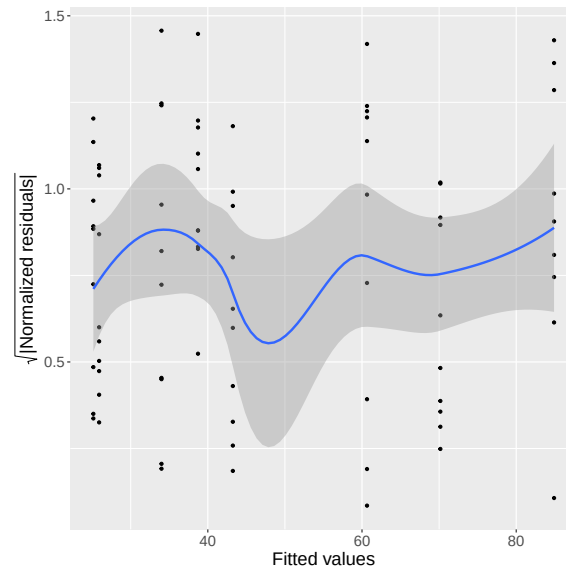
- misspecification of the mean structure

```
plot(eUN.lmm, type = "scatterplot")
```



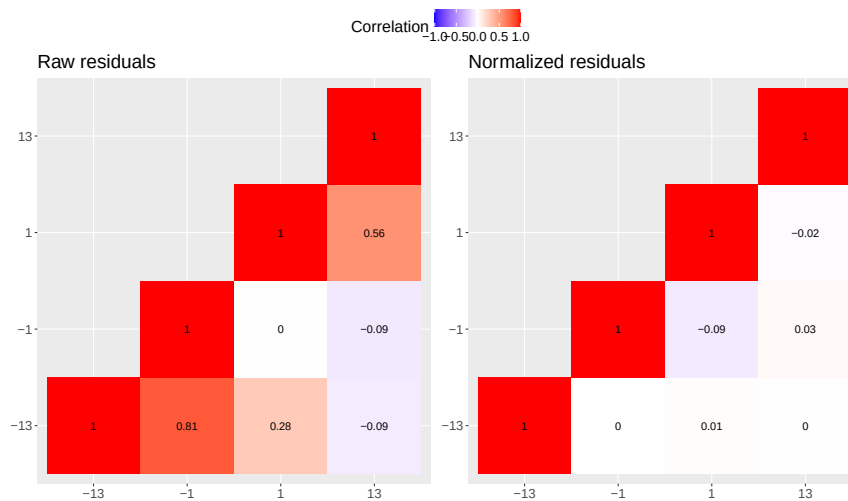
- misspecification of the variance structure

```
plot(eUN.lmm, type = "scatterplot2")
```



- misspecification of the correlation structure

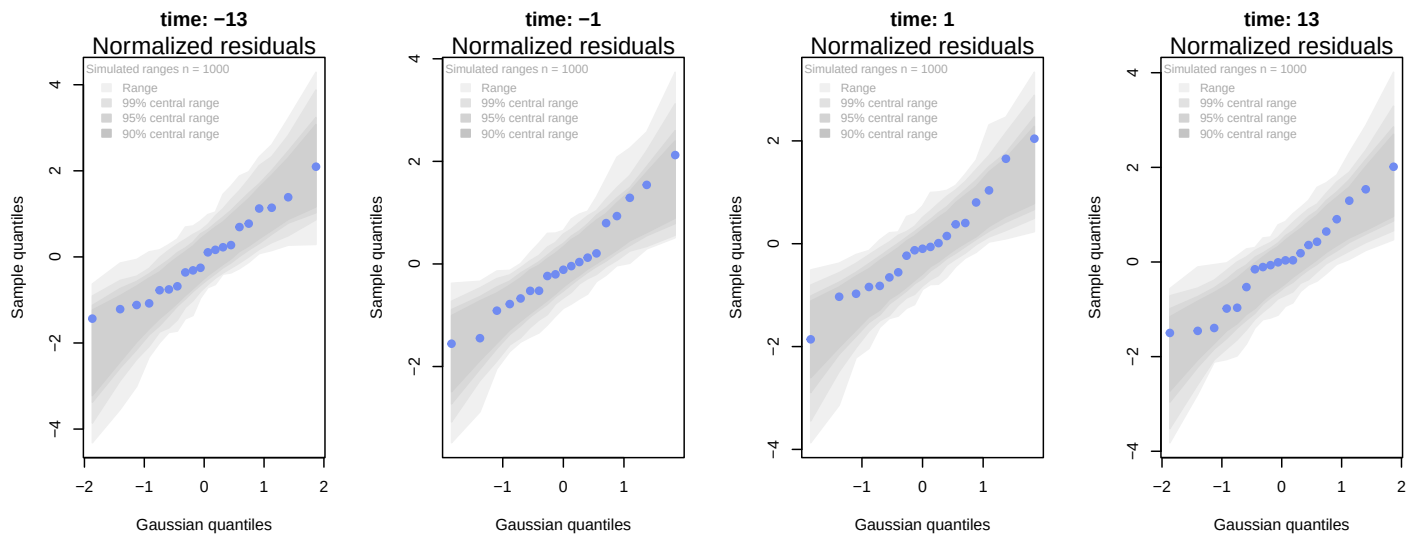
```
plot(eUN.lmm, type = "correlation", type.residual = "response")
plot(eUN.lmm, type = "correlation", type.residual = "normalized")
```



- residual distribution vs. normal distribution ³:

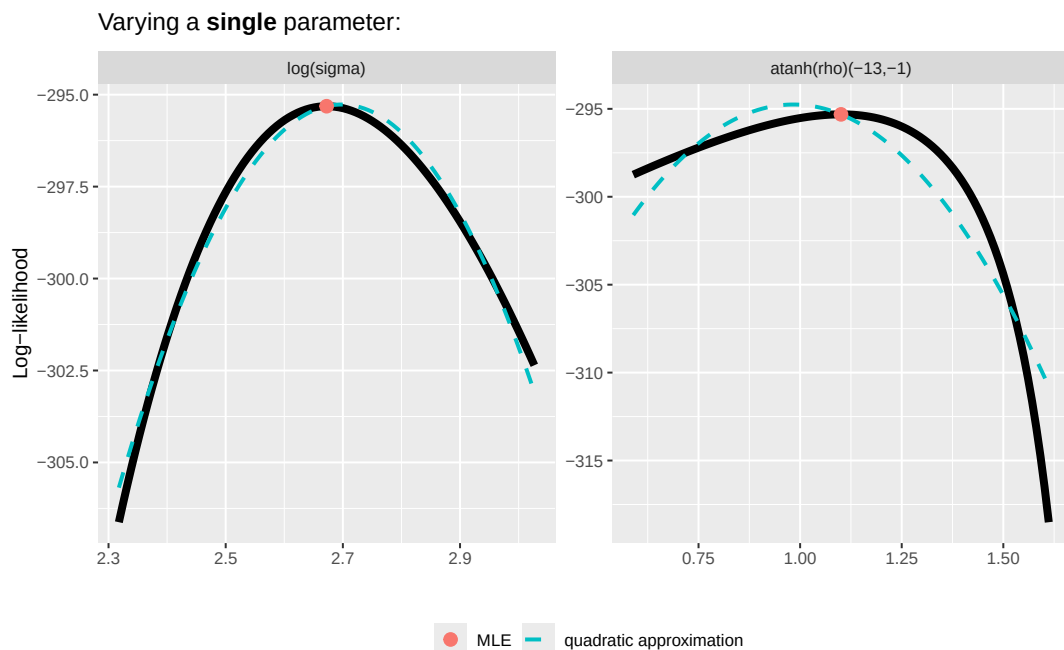
```
plot(eUN.lmm, type = "qqplot", engine = "qqtest",
     facet = ~time, labeller = "label_both", facet_nrow=1)
## Note: the qqtest package to be installed to use the argument engine.plot = "qqtest"
```

³see [Oldford \(2016\)](#) for guidance about how to read quantile-quantile plots.



⚠ Deviation from the normal distribution does not necessarily question the validity of the statistical inference. Moreover, for variance and correlation parameters, normally distributed data is not enough to ensure valid statistical inference. Instead one could assess whether the log-likelihood is locally quadratic as this ensures normally distributed estimates in finite samples (Geyer, 2013). Since the likelihood function is a multi-dimensional function this is not an easy task but one can look at specific 'slices' using the `profile` method:

```
eUN.lmm_profile <- profile(eUN.lmm, effects = c("sigma", "rho(-13,-1)"))
plot(eUN.lmm_profile)
```



6.13 Visualize model fit

The fitted values can be displayed via the `plot` method:

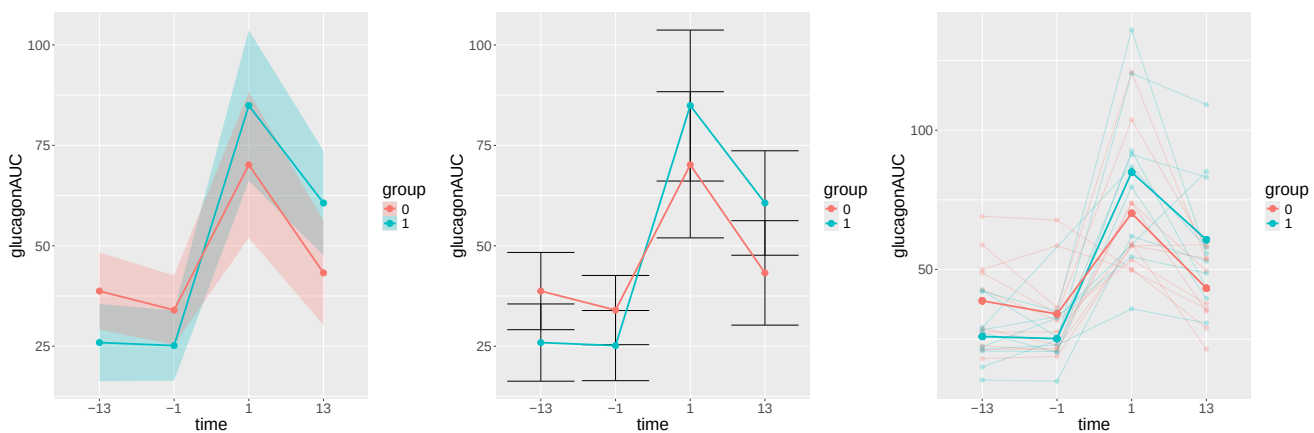
```
## left panel
plot(eUN.lmm, type = "fit", color = "group", size.text = 20)
```

⚠ the shaded area represent 95% confidence intervals (CIs), i.e. is not adjusted for multiplicity over time. More explicit (but sometimes less readable) representation of the CIs can be obtained by setting the argument `ci.alpha` to `NA`:

```
## middle panel
plot(eUN.lmm, type = "fit", color = "group", ci.alpha = NA, size.text = 20)
```

It is also possible to display the observed values along with the fitted values by setting the argument `obs.alpha` to a strictly positive value below or equal to 1. This argument controls the transparency of the color used to display the observed values:

```
## right panel
plot(eUN.lmm, type = "fit", obs.alpha = 0.25, ci = FALSE, size.text = 20)
```



When considering continuous covariates, e.g.:

```
## add baseline weight
gastricbypassLB <- merge(gastricbypassL, gastricbypassW[c("id", "weight1")], by = "id")

eUN.lmmB <- lmm(glucagonAUC ~ weight1 + visit*group, repetition = ~time|id,
  structure = "UN", data = gastricbypassLB)
```

The default graphical display can be confusing as it shows one curve per distinct set of covariate values, i.e. one line per subject:

```
## left panel
plot(eUN.lmmB, type = "fit", color = "group", ci = FALSE, size.text = 20)
```

It is possible to restrict the display specific to a covariate value via the argument `at`:

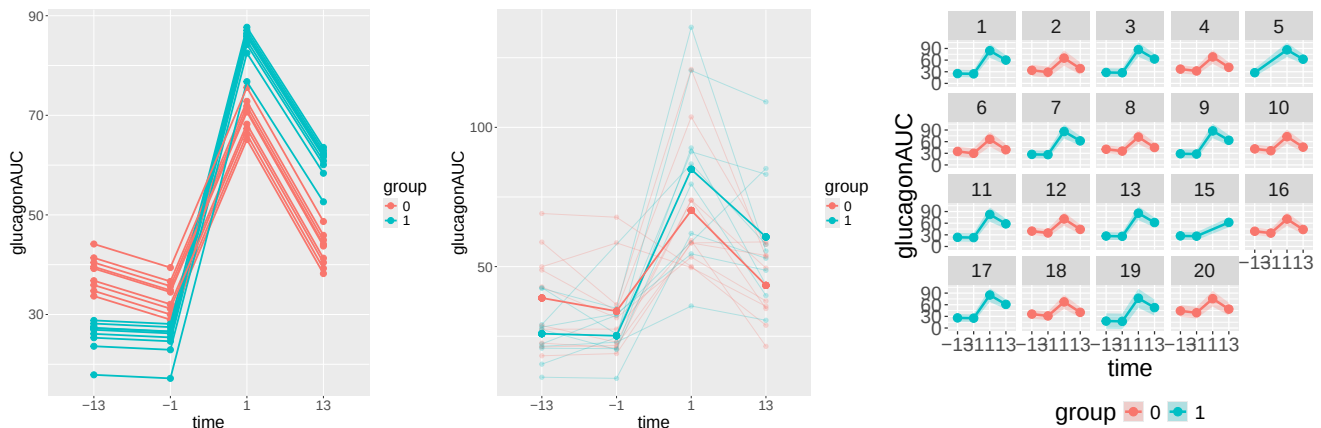
```
## middle panel
plot(eUN.lmmB, type = "fit", color = "group", ci = FALSE, size.text = 20,
  at = data.frame(weight1 = 150), obs.alpha = 0.2)
```

The `plot` method calls the `autoplot` methods which returns a list containing:

- a `ggplot2` object (element `plot`)
- the dataset used to generate the `ggplot2` object (element `data`)

This should ease further customization of the graphical display, e.g.:

```
## right panel
gg.traj <- autoplot(eUN.lmmB, type = "fit", color = "group", size.text = 20, facet = ~id)
gg.traj$plot + theme(legend.position = "bottom")
```



6.14 Partial residuals

In a linear model where we split the covariates and mean parameters into two sets:

$$Y_i = X_{1,i}\beta_1 + X_{2,i}\beta_2 + \varepsilon_i$$

the partial residuals w.r.t. to the covariate(s) X_2 are defined by $\varepsilon_i^{X_2} = Y_i - X_{1,i}\beta_1$.

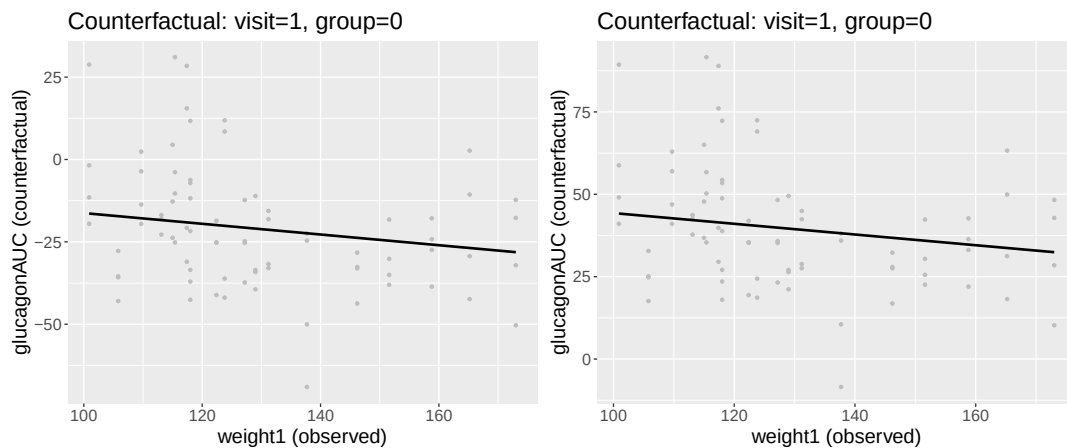
They can be computed via the `residuals` method:

```
df.pres <- residuals(eUN.lmmB, type = "partial", variable = "weight1", keep.data = TRUE)
head(df.pres)
```

	id	visit	time	weight	glucagonAUC	group	baseline	weight1	fitted	r.partial
1	1	1	-13	127.2	20.690	0	TRUE	127.2	-20.684	-25.3242
2	1	1	1	115.5	92.600	0	FALSE	127.2	-20.684	-12.2923
3	1	1	-1	120.7	20.535	0	TRUE	127.2	-20.684	-24.7703
4	1	1	13	108.1	43.434	0	FALSE	127.2	-20.684	-37.3259
5	10	1	13	90.9	57.942	0	FALSE	118.0	-19.188	-7.1423
6	10	1	1	99.3	103.728	0	FALSE	118.0	-19.188	11.7323

In the output, the X_1 covariates (`time` and `group`) have been set to the reference level (-13 and 0) for all observations. Confusion with the ordering variable from the `repetition` argument of `lmm` was avoided by using a different 'time' variable in the mean (`time`) and repetition argument (`visit`) when calling `lmm`. These residuals can be directly displayed via the `plot` method:

```
## left panel
plot(eUN.lmmB, type = "partial", variable = "weight1")
## right panel
plot(eUN.lmmB, type = "partial", variable = c("(Intercept)", "weight1"))
```



The `plot` methods can handle one continuous and one categorical covariate (in addition to the intercept) to display interaction plots. In that case each observation/fitted line is colored according to the categorical covariate.

6.15 Statistical inference (single model, linear)

The `anova` method can be used to test one or several linear combinations of the model coefficients using Wald tests. By default, it will simultaneously test all parameters associated to a variable:

```
anova(eUN.lmm)
```

Multivariate Wald test

	F-statistic	df	p.value
mean: visit	5.803 (3,16.9)		0.00647 **
: group	3.926 (1,18.0)		0.06302 .
: visit:group	2.762 (3,17.3)		0.07332 .

Note that here the p-values are not adjust for multiple comparisons over variables. It is possible to specify a null hypothesis to be test: e.g. is there a change in average weight just after taking the treatment in the reference group:

```
anova(eUN.lmm, effects = c("visit3-visit2=0"))
```

Multivariate Wald test

	F-statistic	df	p.value
all: 1	14.318 (1,17.8)		0.00138 **

One can also simulateneously tests several null hypotheses:

```
e.anova <- anova(eUN.lmm, effects = c("visit3-visit2=0","visit4-visit2=0"))
summary(e.anova)
```

Multivariate Wald test

	F-statistic	df	p.value
all: 1	8.512 (2,17.2)		0.0027 **

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1.

Degrees of freedom were computed using a Satterthwaite approximation (column df).

Univariate Wald test

	estimate	se	df	lower	upper	p.value
visit3 - visit2	36.167	9.558	17.8	13.381	58.953	0.00263 **
visit4 - visit2	9.256	7.738	18	-9.192	27.704	0.38153

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1.

Columns lower/upper/p.value adjusted for multiple comparisons -- max-test.

(1e+05 samples have been used)

Model-based standard errors are derived from the observed information (column se).

Degrees of freedom were computed using a Satterthwaite approximation (column df).

or return all pairwise comparisons for a given factor using the `mcp` function of the `multcomp` package:

```
library(multcomp)
summary(anova(eUN.lmm, effects = mcp(visit = "Tukey")))
```

Singular contrast matrix: contrasts "3 - 2" "4 - 2" "4 - 3" have been removed.

Multivariate Wald test

	F-statistic	df	p.value
all: visit	5.803	(3,16.9)	0.00647 **

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1.

Degrees of freedom were computed using a Satterthwaite approximation (column df).

Univariate Wald test

	estimate	se	df	lower	upper	p.value
2 - 1	-4.734	2.776	17.5	-12.451	2.982	0.32482
3 - 1	31.433	8.63	17.6	7.444	55.422	0.00860 **
4 - 1	4.521	8.005	18	-17.731	26.774	0.93260
3 - 2	36.167	9.558	17.8	9.597	62.737	0.00660 **
4 - 2	9.256	7.738	18	-12.256	30.767	0.60663
4 - 3	-26.912	7.448	16.4	-47.615	-6.209	0.00916 **

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1.

Columns lower/upper/p.value adjusted for multiple comparisons -- max-test.

(1e+05 samples have been used)

Model-based standard errors are derived from the observed information (column se).

Degrees of freedom were computed using a Satterthwaite approximation (column df).

Warning message:

In `mcp2matrix(model, linfct = linfct)` :

covariate interactions found -- default contrast might be inappropriate

Here the `summary` method prints not only the global test but also the result associated to each hypothesis. The warning is triggered by the presence of an interaction between `visit` and `group`: the time effect is only tested here for the reference group. One should look also at the time effect in the other group before concluding about the possible absence of a time effect.

Special characters: special characters, such as parentheses or mathematical operators, can cause problems when using this formula-like interface to specify linear contrasts on parameters. This typically arises when testing (transformed) variance or correlation parameters, parentheses:

```
try(
  anova(eUN.lmm,
    effects = c("log(k).-1=0", "log(k).1=0", "log(k).13=0"))
)
```

```
Error in .anova_Wald(object, effects = effects, robust = robust, multivariate = multivariate, :
Possible misspecification of the argument 'effects' as running mulcomp::glht lead to the following
Error in parse(text = ex[i]) : <text>:1:7: unexpected symbol
1: log(k).
~
```

It is then advised to build a contrast matrix, e.g.:

```
name.coef <- rownames(confint(eUN.lmm, effects = "all"))
name.varcoef <- grep("^k",name.coef, value = TRUE)
C <- matrix(0, nrow = 3, ncol = length(name.coef), dimnames = list(name.varcoef, name.coef))
diag(C[name.varcoef,name.varcoef]) <- 1
C[,1:9]
```

	(Intercept)	visit2	visit3	visit4	group1	visit2:group1	visit3:group1	visit4:group1	sigma
k.-1	0	0	0	0	0	0	0	0	0
k.1	0	0	0	0	0	0	0	0	0
k.13	0	0	0	0	0	0	0	0	0

And then call the `anova` method specifying the null hypothesis via the contrast matrix:

```
anova(eUN.lmm, effects = C)
```

Multivariate Wald test

	F-statistic	df	p.value
all: 1	3.388	(3,25.7)	0.0332 *

6.16 Statistical inference (multiple models, linear)

It is possible to adjust for multiple testing across several linear contrasts that may originate from different `lmm` using the approach of [Pipper et al. \(2012\)](#):

- fit the mixed models using `lmm`. The LMM must be fitted on the same dataset (or on subsets on a common larger dataset) with the same `repetition` argument.
- use the `anova` method to indicate which hypotheses are being tested
- combine the tests using `rbind`.

Here is an (artificial) example:

```
Manova <- rbind(anova(eInd.lmm, effects = "visit3:group1 = 0", robust = FALSE),
               anova(eCS.lmm, effects = "visit3:group1 = 0", robust = FALSE),
               anova(eUN.lmm, effects = "visit3:group1 = 0", robust = FALSE),
               name = c("Ind", "CS", "UN"))
summary(Manova)
```

Multivariate Wald test

	Chi2-statistic	df	p.value
all: 1	116.9	(3,Inf)	<1e-04 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1.

Univariate Wald test

	estimate	se	df	lower	upper	p.value
Ind: visit3:group1	27.001	14.285	25.3	-1.482	55.485	0.0631 .
CS: visit3:group1	27.481	11.09	52.8	5.369	49.593	0.0137 *
UN: visit3:group1	27.571	12.42	17.8	2.808	52.335	0.0268 *

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1.

Columns lower/upper/p.value adjusted for multiple comparisons -- max-test.

(error when computing the adjusted columns lower/upper/p.value by numerical integration: 0.00073)

Model-based standard errors are derived from the observed information (column se).

6.17 Statistical inference (single model, non-linear)

The `estimate` function can be used to test one or several non-linear combinations of model coefficients, using a first order delta method to quantify uncertainty. The combination has to be specified via a function (argument `f`). To illustrate its use consider an ANCOVA analysis:

$$Y_{i1} = \alpha + \beta Y_{i,0} + \gamma X_i + e_i$$

```
e.ANCOVA <- lm(weight4 ~ weight1 + group, data = gastricbypassW)
summary(e.ANCOVA)$coef
```

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	-5.92851	8.780064	-0.67522	5.0861e-01
weight1	0.82363	0.064116	12.84598	3.5247e-10
group	4.14046	2.533355	1.63438	1.2056e-01

We can replicate this analysis by first fitting a mixed model:

$$Y_{ij} = \alpha_j + \gamma_j X_i + \varepsilon_{i,j} \text{ where } \varepsilon_i \sim \mathcal{N} \left(\begin{bmatrix} 0 \\ 0 \end{bmatrix}, \begin{bmatrix} \sigma_1^2 & \rho\sigma_1\sigma_2 \\ \rho\sigma_1\sigma_2 & \sigma_2^2 \end{bmatrix} \right)$$

```
gastricbypassL14 <- gastricbypassL[gastricbypassL$visit %in% c(1,4),]
gastricbypassL14$visit <- droplevels(gastricbypassL14$visit)
e.lmmANCOVA <- lmm(weight ~ visit + visit:group, repetition = ~visit|id,
  data = gastricbypassL14)
```

and then perform a first order delta-method:

```
lava::estimate(e.lmmANCOVA, f = function(p){
  c(Y1 = as.double(p["rho(1,4)"]*p["k.4"]),
    X1 = as.double(p["visit4:group1"]-p["rho(1,4)"]*p["k.4"]*p["visit1:group1"]))
})
```

	estimate	se	df	lower	upper	p.value
Y1	0.82363	0.062309	9.8746	0.68456	0.9627	1.3327e-07
X1	4.14046	2.461978	15.1613	-1.10227	9.3832	1.1309e-01

Indeed:

$$\begin{aligned} \mathbb{E}[Y_{i2}|Y_{i1}, X_i] &= \alpha_2 + \gamma_2 X_i + \rho \frac{\sigma_2}{\sigma_1} (Y_{i1} - \alpha_1 - \gamma_1 X_i) \\ &= \alpha_2 - \rho \frac{\sigma_2}{\sigma_1} \alpha_1 + \rho \frac{\sigma_2}{\sigma_1} Y_{i1} + \left(\gamma_2 - \rho \frac{\sigma_2}{\sigma_1} \gamma_1 \right) X_i \end{aligned}$$

We obtain identical estimate but different standard-errors/degrees of freedom compared to the univariate linear model approach. The later is to be prefer as it does not rely on approximation. The former is nevertheless useful as it can handle missing data in the outcome variable.

6.18 Baseline adjustment

In clinical trial the group and intervention variable often do not coincide, e.g., in presence of baseline measurement. In our running example, the first two measurement are pre-treatment (i.e. treatment should be "none") while the last two measurements are post-treatment (i.e. treatment should be 1 or 2). The `baselineAdjustment` function can be helpful to define a time varying treatment variable:

- where baseline takes a specific value

```
gastricbypassL$treat <- baselineAdjustment(gastricbypassL, variable = "group",
                                           repetition = ~visit|id, constrain = c("1","2"),
                                           new.level = "none")
table(treat = gastricbypassL$treat,
      visit = gastricbypassL$visit,
      group = gastricbypassL$group)
```

```
, , group = 0
```

	visit			
treat	1	2	3	4
none	10	10	0	0
0	0	0	10	10
1	0	0	0	0

```
, , group = 1
```

	visit			
treat	1	2	3	4
none	10	10	0	0
0	0	0	0	0
1	0	0	10	10

- where baseline corresponds to the reference group

```
gastricbypassL$treat2 <- baselineAdjustment(gastricbypassL, variable = "group",
                                             repetition = ~visit|id, constrain = c("1","2"))
table(treat = gastricbypassL$treat2,
      visit = gastricbypassL$visit,
      group = gastricbypassL$group)
```

```
, , group = 0
```

	visit			
treat	1	2	3	4
0	10	10	10	10
1	0	0	0	0

```
, , group = 1
```

	visit			
treat	1	2	3	4
0	10	10	0	0
1	0	0	10	10

- including interactions with group

```
gastricbypassL$visitXtreat <- baselineAdjustment(gastricbypassL, variable = "group",
                                                  repetition = ~visit|id, constrain = c("1","2"),
                                                  collapse.time = ".")
table(treat = gastricbypassL$visitXtreat,
      visit = gastricbypassL$visit,
      group = gastricbypassL$group)
```

```
, , group = 0
```

```
      visit
treat 1  2  3  4
  1   10  0  0  0
  2    0 10  0  0
 3.0   0  0 10  0
 4.0   0  0  0 10
 3.1   0  0  0  0
 4.1   0  0  0  0
```

```
, , group = 1
```

```
      visit
treat 1  2  3  4
  1   10  0  0  0
  2    0 10  0  0
 3.0   0  0  0  0
 4.0   0  0  0  0
 3.1   0  0 10  0
 4.1   0  0  0 10
```

We would then typically like to model group differences only after baseline (i.e. only at 1 week and 3 months after). This can be performed using the time varying treatment variable, e.g.:

```
eC.lmm <- lmm(glucagonAUC ~ visitXtreat, data = gastricbypassL,
              repetition = ~visit|id, structure = "UN")
coef(eC.lmm) ## change from baseline
```

```
(Intercept)   visitXtreat2 visitXtreat3.0 visitXtreat4.0 visitXtreat3.1 visitXtreat4.1
 32.3167      -2.7478      34.3703      11.6559      56.0581      27.6108
```

or

```
eC2.lmm <- lmm(glucagonAUC ~ 0 + visitXtreat, data = gastricbypassL,
               repetition = ~visit|id, structure = "UN")
coef(eC2.lmm) ## absolute value
```

```
visitXtreat1   visitXtreat2 visitXtreat3.0 visitXtreat4.0 visitXtreat3.1 visitXtreat4.1
 32.317        29.569        66.687        43.973        88.375        59.927
```

The parametrization however does not (directly) output treatment effects. Instead one may be tempted to use a formula like `treatment*time`. However this will lead to a non-identifiable model. Indeed we are only able to estimate a total of 6 means when constraining the expected baseline value between the two groups to be the same. Therefore can at most identify 6 effects. However the design matrix for the interaction model:

```
colnames(model.matrix(glucagonAUC ~ treat*visit, data = gastricbypassL))
```

```
[1] "(Intercept)" "treat0"      "treat1"      "visit2"      "visit3"      "visit4"
[7] "treat0:visit2" "treat1:visit2" "treat0:visit3" "treat1:visit3" "treat0:visit4" "treat1:visit4"
```

contains 12 parameters (i.e. 6 too many). Fortunately, the lmm will drop non-identifiable effects from the model and fit the resulting simplified model:

```
eC3.lmm <- lmm(glucagonAUC ~ treat2*visit, data = gastricbypassL,
               repetition = ~visit|id, structure = "UN")
```

Constant values in the design matrix for the mean structure.

Coefficients "treat21" "treat21:visit2" relative to interactions "treat2:visit" have been removed.

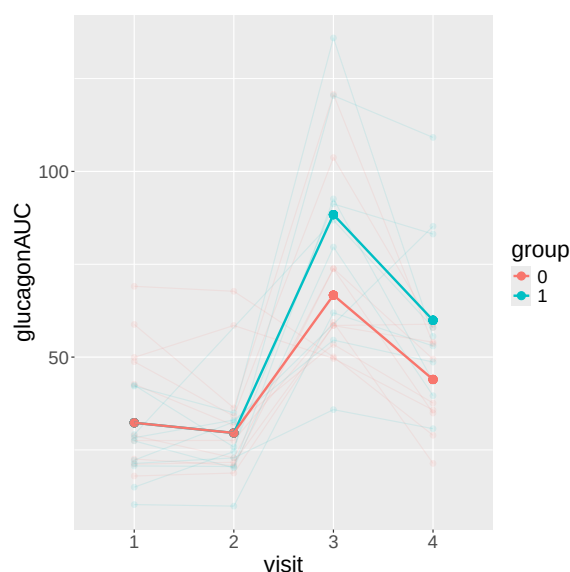
with the following coefficients:

```
model.tables(eC3.lmm)
```

	estimate	se	df	lower	upper	p.value
(Intercept)	32.3167	3.4764	19.003	25.0407	39.5927	1.6802e-08
visit2	-2.7478	1.9950	19.007	-6.9232	1.4276	1.8441e-01
visit3	34.3703	8.6111	15.161	16.0331	52.7076	1.1573e-03
visit4	11.6559	7.5601	17.328	-4.2715	27.5833	1.4119e-01
treat21:visit3	21.6878	12.2967	13.748	-4.7316	48.1072	9.9982e-02
treat21:visit4	15.9549	9.6174	12.374	-4.9296	36.8395	1.2224e-01

One can visualize the baseline adjustment via the `plot` function:

```
plot(eC3.lmm, color = "group", ci = FALSE, size.text = 20, obs.alpha = 0.1)
```



and retrieve the treatment at each timepoint using the `effects` method:

```
effects(eC3.lmm, variable = "treat2", type = "difference")
```

Difference in average counterfactual outcome
w.r.t 'treat2' values

	estimate	se	df	lower	upper	p.value
treat2=1-0(t=1)	0	0	Inf	0	0	NA
treat2=1-0(t=2)	0	0	Inf	0	0	NA
treat2=1-0(t=3)	21.688	12.297	13.7	-4.732	48.107	0.100 .
treat2=1-0(t=4)	15.955	9.617	12.4	-4.93	36.84	0.122

6.19 Predictions

Two types of predictions can be performed with the `predict` method:

- **static predictions** that are only conditional on the covariates:

```
news <- gastricbypassL[gastricbypassL$id==2,]  
news$glucagon <- 0  
predict(eUN.lmm, newdata = news, se = TRUE)
```

```
      estimate      se      df lower upper  
1  38.729 4.5765 18.003 29.114 48.344  
2  33.995 4.1002 17.897 25.377 42.612  
3  70.162 8.6491 17.695 51.968 88.356  
4  43.250 6.1883 18.005 30.249 56.251
```

which can be computing by creating a design matrix:

```
X.12 <- model.matrix(formula(eUN.lmm), news)  
X.12
```

```
      (Intercept) visit2 visit3 visit4 group1 visit2:group1 visit3:group1 visit4:group1  
2              1      0      0      0      0              0              0              0  
22             1      1      0      0      0              0              0              0  
42             1      0      1      0      0              0              0              0  
62             1      0      0      1      0              0              0              0  
attr(,"assign")  
[1] 0 1 1 1 2 3 3 3  
attr(,"contrasts")  
attr(,"contrasts")$visit  
[1] "contr.treatment"  
  
attr(,"contrasts")$group  
[1] "contr.treatment"
```

and then multiplying it with the regression coefficients:

```
X.12 %*% coef(eUN.lmm)
```

```
      [,1]  
2  38.729  
22 33.995  
42 70.162  
62 43.250
```

- **dynamic predictions** that are conditional on the covariates and the outcome measured at other timepoints. Consider two subjects for who we would like to predict the weight 1 week before the intervention based on the weight 3 months before the intervention:

```
newd <- rbind(
  data.frame(id = 1, time = -13, visit = "1", group = 0, glucagonAUC = coef(eUN.lmm)["(Intercept)"]),
  data.frame(id = 1, time = 1, visit = "3", group = 0, glucagonAUC = NA),
  data.frame(id = 2, time = -13, visit = "1", group = 0, glucagonAUC = 50),
  data.frame(id = 2, time = 1, visit = "3", group = 0, glucagonAUC = NA)
)
predict(eUN.lmm, newdata = newd, type = "dynamic", keep.data = TRUE)
```

	id	time	visit	group	glucagonAUC	estimate	se	df	lower	upper
1	1	-13	1	0	38.729	NA	NA	NA	NA	NA
2	1	1	3	0	NA	70.162	8.3308	17.592	52.630	87.694
3	2	-13	1	0	50.000	NA	NA	NA	NA	NA
4	2	1	3	0	NA	75.888	9.6360	12.711	55.022	96.753

The first subjects starts with the average glucagon while the second starts with a much higher glucagon. The predicted glucagon after the operation for the first subject is then the average glucagon while it is predicted to be higher for the second subject due to the positive correlation over time. The predicted value is computed using the formula of the conditional mean for a Gaussian vector:

```
mu1 <- coef(eUN.lmm)["(Intercept)"]
mu3 <- mu1 + coef(eUN.lmm)["visit3"]
Omega_11 <- sigma(eUN.lmm)[1,1]
Omega_31 <- sigma(eUN.lmm)[3,1]
as.double(mu3 + Omega_31 * (50 - mu1) / Omega_11)
```

```
[1] 75.888
```

7 Equivalence with other statistical methods

7.1 Welch two sample t-test

A two sample t-test:

```
t.test(weight4 ~ group, data = gastricbypassW)
```

Welch Two Sample t-test

```
data: weight4 by group
t = 0.591, df = 17.7, p-value = 0.56
alternative hypothesis: true difference in means between group 0 and group 1 is not equal to 0
95 percent confidence interval:
 -11.736  20.916
sample estimates:
mean in group 0 mean in group 1
      104.66      100.07
```

is equivalent to an independent covariance pattern with a different variable for each group:

```
e.ttest4 <- lmm(weight4 ~ group, structure = IND(~group),
               data = gastricbypassW, trace = FALSE)
model.tables(e.ttest4)
```

	estimate	se	df	lower	upper	p.value
(Intercept)	104.66	5.1045	9.0018	93.113	116.207	7.2710e-09
group	-4.59	7.7607	17.6824	-20.916	11.736	5.6171e-01

7.2 Paired t-test

With complete data, a paired t-test:

```
t.test(gastricbypassW$weight4, gastricbypassW$weight1, paired = TRUE)
```

Paired t-test

```
data: gastricbypassW$weight4 and gastricbypassW$weight1
t = -17.2, df = 19, p-value = 5e-13
alternative hypothesis: true mean difference is not equal to 0
95 percent confidence interval:
 -29.848 -23.362
sample estimates:
mean difference
 -26.605
```

is equivalent to a LMM with an unstructured covariate pattern:

```
e.lmm2tt <- lmm(weight ~ visit, repetition = ~visit|id, structure = "UN",
               data = gastricbypassL)
model.tables(e.lmm2tt)["visit4",,drop=FALSE]
```

	estimate	se	df	lower	upper	p.value
visit4	-26.605	1.5494	18.964	-29.848	-23.362	5.1692e-13

7.3 Welch two sample t-test on the change

With complete data, a two sample t-test comparing the change from baseline:

```
gastricbypassW.0 <- gastricbypassW[gastricbypassW$group==0,]  
gastricbypassW.1 <- gastricbypassW[gastricbypassW$group==1,]  
t.test(gastricbypassW.0$weight4-gastricbypassW.0$weight1,  
       gastricbypassW.1$weight4-gastricbypassW.1$weight1)
```

Welch Two Sample t-test

```
data: gastricbypassW.0$weight4 - gastricbypassW.0$weight1 and gastricbypassW.1$weight4 - gastricby  
t = -2.11, df = 13, p-value = 0.055  
alternative hypothesis: true difference in means is not equal to 0  
95 percent confidence interval:  
 -12.16771  0.14771  
sample estimates:  
mean of x mean of y  
 -29.61    -23.60
```

is equivalent to a LMM with a stratified unstructured covariate pattern:

```
e.lmm2tt2 <- lmm(weight ~ visit*group, repetition = ~visit|id, structure = UN(~group),  
                data = gastricbypassL)  
model.tables(e.lmm2tt2)["visit4:group1",,drop=FALSE]
```

	estimate	se	df	lower	upper	p.value
visit4:group1	6.01	2.8511	13.009	-0.14908	12.169	0.055

7.4 Multiple Student's t-test

Multiple t-tests:

```
e.ttest1 <- lmm(weight1 ~ group, structure = IND(~group),
               data = gastricbypassW, trace = FALSE)
e.ttest2 <- lmm(weight2 ~ group, structure = IND(~group),
               data = gastricbypassW, trace = FALSE)
e.ttest3 <- lmm(weight3 ~ group, structure = IND(~group),
               data = gastricbypassW, trace = FALSE)
```

can be adjusted for multiple comparison by first using the `anova` function to specify the parameter of interest and combining the results using `rbind`:

```
e.mttest <- rbind(anova(e.ttest1, effects = "group=0"),
                  anova(e.ttest2, effects = "group=0"),
                  anova(e.ttest3, effects = "group=0"),
                  anova(e.ttest4, effects = "group=0"))
model.tables(e.mttest, method = "bonferroni")
```

	estimate	se	df	lower	upper	p.value
weight1: group	-10.60	8.9717	17.965	-35.498	14.298	1
weight2: group	-9.50	8.3951	17.985	-32.795	13.795	1
weight3: group	-8.92	8.1295	17.959	-31.481	13.641	1
weight4: group	-4.59	7.7607	17.682	-26.165	16.985	1

⚠ efficient adjustment for multiple comparisons (like "single-step") will not be valid as the correlation structure has not been specified. To do so it is more convenient to work with a the long format:

```
e.mttest2 <- mlmm(weight ~ group, structure = IND(~group),
                 data = gastricbypassL, trace = FALSE,
                 effects = "group1=0", by = "time", repetition = ~time|id)
model.tables(e.mttest2, method = "single-step2")
```

	by	parameter	estimate	se	df	lower	upper	p.value
1	-13	group1	-10.60	8.9717	17.965	-30.989	9.7893	0.31768
2	-1	group1	-9.50	8.3951	17.985	-28.579	9.5789	0.34123
3	1	group1	-8.92	8.1295	17.959	-27.395	9.5551	0.35785
4	13	group1	-4.59	7.7607	17.682	-22.227	13.0470	0.66473

or call the dedicated function `mt.test`:

```
mt.test(weight1+weight2+weight3+weight4~group, data = gastricbypassW)
```

	by	parameter	estimate	se	df	lower	upper	p.value
1	weight1	group	-10.60	8.9717	17.965	-30.976	9.7758	0.31870
2	weight2	group	-9.50	8.3951	17.985	-28.566	9.5663	0.34115
3	weight3	group	-8.92	8.1295	17.959	-27.383	9.5429	0.35844
4	weight4	group	-4.59	7.7607	17.682	-22.215	13.0354	0.66272

7.5 Linear regression on the change

A widely spread approach to analyze longitudinal data is to reduce the number of repetitions to 1 by working on the change and then apply 'usual' statistical methods. For instance one could compare the pre- and post- operation values using:

```
gastricbypassW$changeG41 <- gastricbypassW$glucagonAUC4-gastricbypassW$glucagonAUC1
e.change41 <- lm(changeG41 ~ weight1, data = gastricbypassW)
summary(e.change41)$coef
```

```
              Estimate Std. Error t value Pr(>|t|)
(Intercept)  88.41370   41.01024   2.1559 0.044871
weight1      -0.53331    0.31432  -1.6967 0.106975
```

This turns out to be equivalent to the following mixed model:

```
gastricbypassL41 <- gastricbypassL[gastricbypassL$visit %in% c(1,4),]
gastricbypassL41$visit <- droplevels(gastricbypassL41$visit)
gastricbypassL41$weight1 <- gastricbypassW$weight1[gastricbypassL41$id]

e.lmm41 <- lmm(glucagonAUC ~ visit + visit*weight1,
               repetition =~ visit|id, structure = "UN",
               data = gastricbypassL41)
model.tables(e.lmm41)
```

```
              estimate      se    df    lower    upper p.value
(Intercept)   31.7805917 23.58747 18.003 -17.77425  81.33543 0.19458
visit4        88.4137014 41.01024 18.001   2.25477 174.57264 0.04487
weight1        0.0041566  0.18078 18.003  -0.37565   0.38396 0.98191
visit4:weight1 -0.5333052  0.31432 18.001  -1.19366   0.12705 0.10697
```

This equivalence only holds as there is no missing data.

```
index.missing41 <- which(is.na(gastricbypassW$changeG41))
index.missing41
```

```
integer(0)
```

7.6 Correlation between changes

In some studies, one is interested in studying the relation between two evolutions. Say weight and glucagon before and after the operation:

```
gastricbypassW$changeG41 <- gastricbypassW$glucagonAUC4-gastricbypassW$glucagonAUC1
gastricbypassW$changeW41 <- gastricbypassW$weight4-gastricbypassW$weight1
```

One can evaluate their correlation:

```
cor.test(gastricbypassW$changeW41, gastricbypassW$changeG41)
```

Pearson's product-moment correlation

```
data: gastricbypassW$changeW41 and gastricbypassW$changeG41
t = 1.89, df = 18, p-value = 0.075
alternative hypothesis: true correlation is not equal to 0
95 percent confidence interval:
 -0.043829  0.719624
sample estimates:
      cor
0.40658
```

or regress one against the other:

```
e2.change41 <- lm(changeG41 ~ changeW41, data = gastricbypassW)
summary(e2.change41)$coef
```

```
              Estimate Std. Error t value Pr(>|t|)
(Intercept)  65.0794    24.83368   2.6206 0.017331
changeW41     1.7082     0.90473   1.8881 0.075246
```

This problem can be recast using all measurement as outcomes:

```
keep.col <- c("id","weight1","weight4","glucagonAUC1","glucagonAUC4")
gastricbypassL4 <- reshape(gastricbypassW[,keep.col], direction = "long",
                           idvar = "id", varying = 2:5, timevar = "type", v.names = "value")
gastricbypassL4$type <- factor(gastricbypassL4$type, labels = keep.col[-1])
gastricbypassL4 <- gastricbypassL4[order(gastricbypassL4$id),]
head(gastricbypassL4)
```

```
      id      type  value
1.1  1    weight1 127.200
1.2  1    weight4 108.100
1.3  1 glucagonAUC1  20.690
1.4  1 glucagonAUC4  43.434
2.1  2    weight1 165.200
2.2  2    weight4 132.000
```

fitting an unstructured mixed model:

```
e.lmm4 <- lmm(value ~ type,  
              repetition = ~type|id, structure = "UN",  
              data = gastricbypassL4)
```

extract the residual covariance matrix:

```
sigma.lmm4 <- sigma(e.lmm4)  
sigma.lmm4
```

```
              weight1 weight4 glucagonAUC1 glucagonAUC4  
weight1      410.8475  326.84      1.7077    -217.399  
weight4      326.8357  290.84     -24.6003    -161.696  
glucagonAUC1  1.7077  -24.60     241.7007     -81.649  
glucagonAUC4 -217.3994 -161.70    -81.6493     442.464
```

Deduce the residual covariance matrix for the change:

```
Mcon <- cbind(c(-1,1,0,0),c(0,0,-1,1))  
sigmeChange.lmm4 <- t(Mcon) %*% sigma.lmm4 %*% Mcon  
dimnames(sigmeChange.lmm4) <- list(c("d.weight","d.glucagonAUC"),  
                                   c("d.weight","d.glucagonAUC"))  
sigmeChange.lmm4
```

```
              d.weight d.glucagonAUC  
d.weight      48.011      82.011  
d.glucagonAUC 82.011      847.464
```

and the correlation or covariance:

```
cov2cor(sigmeChange.lmm4)[1,2]  
sigmeChange.lmm4[1,2]/sigmeChange.lmm4[1,1]
```

```
[1] 0.40658  
[1] 1.7082
```

The uncertainty can be quantified using a delta method:

```
estimate(e.lmm4, function(p){  
  Sigma.change <- t(Mcon) %*% sigma(e.lmm4, p = p) %*% Mcon  
  c(cor = cov2cor(Sigma.change)[1,2],  
    beta = Sigma.change[1,2]/Sigma.change[1,1])  
})
```

```
      estimate      se      df      lower upper p.value  
cor    0.40658 0.19150 2.5925 -0.26078 1.0739 0.13791  
beta   1.70818 0.88073 2.6876 -1.28836 4.7047 0.15837
```

The standard errors and degrees of freedom do not match the univariate analysis, suggesting poor small sample properties of this technic.

8 Missing values and imputation

We reconsider the example of the previous section, but now in presence of missing values. The `summarize` function can be used to describe the amount of missing data at each repetition:

```
sss <- summarize(glucagonAUC ~ time, data = gastricbypassL, na.rm = TRUE)
cbind(sss[,1:4], pc = paste0(100 * sss$missing / (sss$missing + sss$observed), "%"))
```

```
      outcome time observed missing pc
1 glucagonAUC  -13      20       0 0%
2 glucagonAUC   -1      19       1 5%
3 glucagonAUC    1      19       1 5%
4 glucagonAUC   13      20       0 0%
```

For more detail about the missing data patterns, see the `summarizeNA` function:

```
summarizeNA(data = gastricbypassL, repetition = ~ time|id)
```

```
variable frequency missing.pattern n.missing id -13 -1 1 13
      visit         20          00000         0 0 0 0 0 0
      weight         20          00000         0 0 0 0 0 0
glucagonAUC         18          00000         0 0 0 0 0 0
              1          00010         1 0 0 0 1 0
              1          00100         1 0 0 1 0 0
      group         20          00000         0 0 0 0 0 0
baseline         20          00000         0 0 0 0 0 0
      treat         20          00000         0 0 0 0 0 0
      treat2        20          00000         0 0 0 0 0 0
timeXtreat        20          00000         0 0 0 0 0 0
visitXtreat       20          00000         0 0 0 0 0 0
      group2        20          00000         0 0 0 0 0 0
```

To begin with we will only consider 1 week before and 1 week after surgery:

```
## long format
gastricbypassL32 <- gastricbypassL[gastricbypassL$visit %in% c(3,2),]
gastricbypassL32$visit <- droplevels(gastricbypassL32$visit)
gastricbypassL32$weight1 <- gastricbypassW$weight1[gastricbypassL32$id]
## wide format
gastricbypassW$changeG32 <- gastricbypassW$glucagonAUC3-gastricbypassW$glucagonAUC2
```

8.1 Full information approach

LMM uses a full information approach:

```
e.lmm32 <- lmm(glucagonAUC ~ visit + visit*weight1,
              repetition =~ visit|id, structure = "UN",
              data = gastricbypassL32)
model.tables(e.lmm32)
```

	estimate	se	df	lower	upper	p.value
(Intercept)	9.24975	20.66248	17.011	-34.34202	52.84153	0.66004717
visit3	170.75489	39.66297	17.808	87.36171	254.14806	0.00043568
weight1	0.15750	0.15734	17.011	-0.17444	0.48945	0.33083625
visit3:weight1	-0.95238	0.30205	17.648	-1.58787	-0.31688	0.00560372

whereas a linear model would perform a complete case approach:

```
e.change32 <- lm(changeG32 ~ weight1, data = gastricbypassW)
summary(e.change32)$coef
```

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	173.46620	41.75201	4.1547	0.00074599
weight1	-0.96982	0.31589	-3.0701	0.00732363

In the former the likelihood is evaluated using all observations, even those from individuals with some (but not all) missing outcome values: baseline is used even if follow-up is missing. In the later the likelihood is only evaluated on individuals with no missing outcome values: if follow-up is missing then baseline is not used. Indeed:

```
coef(lm(changeG32 ~ weight1, data = gastricbypassW[-c(5,15),]))
```

(Intercept)	weight1
173.46620	-0.96982

The estimates of the LMM can be retrived using a linear model where we have imputed the conditional expectation of the missing values given the observed value and the estimated model parameters: (see section 8.3 for a graphical representation)

```
gastricbypassWA <- fitted(e.lmm32, type = "outcome", format = "wide")
gastricbypassWA$change32 <- gastricbypassWA$glucagonAUC_3 - gastricbypassWA$glucagonAUC_2
gastricbypassWA$weight1 <- gastricbypassW$weight1[match(gastricbypassW$id,gastricbypassWA$id)]
coef(lm(change32 ~ weight1, data = gastricbypassWA))
```

(Intercept)	weight1
170.75489	-0.95238

 Standard errors, confidence intervals, and p-values from this linear model should not be trusted as they do not account for the uncertainty in the imputed values.

8.2 Complete case approach

The `lmmCC` can be used to obtain the LMM that is equivalent to a linear regression. In the case of the comparing the change between groups, the `repetition` argument should indicate how the change has been computed:

```
e.lmmCC <- lmmCC(e.change32, repetition = changeG32 ~ glucagonAUC3-glucagonAUC2|id)
model.tables(e.lmmCC)
```

Remove 2 clusters (4 observations)

- 2 observations with missing data (2 clusters)

- 0 missing repetitions (0 clusters)

	estimate	se	df	lower	upper	p.value
(Intercept)	-165.90910	55.22956	16	-282.99061	-48.82760	0.00840925
time	173.46620	41.75201	16	84.95611	261.97630	0.00074594
weight1	1.13813	0.41786	16	0.25231	2.02395	0.01502328
time:weight1	-0.96982	0.31589	16	-1.63948	-0.30017	0.00732343

As output, the data from two clusters (i.e. 4 observations) has been excluded before fitting the LMM (instead of just the 2 observations with missing values for the full information approach). The interaction term of the LMM matches the regression coefficient of the linear model:

```
summary(e.change32)$coef
```

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	173.46620	41.75201	4.1547	0.00074599
weight1	-0.96982	0.31589	-3.0701	0.00732363

In the case of regressing two changes:

```
gastricbypassW$changeW32 <- gastricbypassW$weight3 - gastricbypassW$weight2
```

```
e2g.change32 <- lm(changeG32 ~ changeW32 + group, data = gastricbypassW)
summary(e2g.change32)$coef
```

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	16.998	30.9485	0.54924	0.59093
changeW32	-3.288	5.0127	-0.65594	0.52180
group	26.361	15.7415	1.67463	0.11472

the `repetition` argument should indicate how each change has been computed:

```
e2.lmmCC <- lmmCC(e2g.change32, repetition = list(changeG32 ~ glucagonAUC3-glucagonAUC2|id,
                                                  changeW32 ~ weight3-weight2|id))
model.tables(e2.lmmCC)
```

```
Remove 2 clusters (8 observations)
- 2 observations with missing data (2 clusters)
- 0 missing repetitions (0 clusters)
      estimate      se      df      lower      upper p.value
cor  -0.16699 0.24255 1.7257  -1.3868   1.0529 0.57192
beta -3.28804 4.83126 2.3131 -21.5987 15.0226 0.55791
```

We retrieve the same estimate for the effect of change in weights but the uncertainty (standard error, confidence intervals, p.value) do not match. They should be asymptotically correct but may not have very good small sample properties.

8.3 Imputation

When fitting a linear mixed model on a dataset with missing values:

```
eUN.lmmNA <- lmm(glucagonAUC ~ time, repetition = ~time|id, data = gastricbypassL)
nobs(eUN.lmmNA)
```

```
obs      cluster  missing.obs missing.cluster
78         20           2             0
```

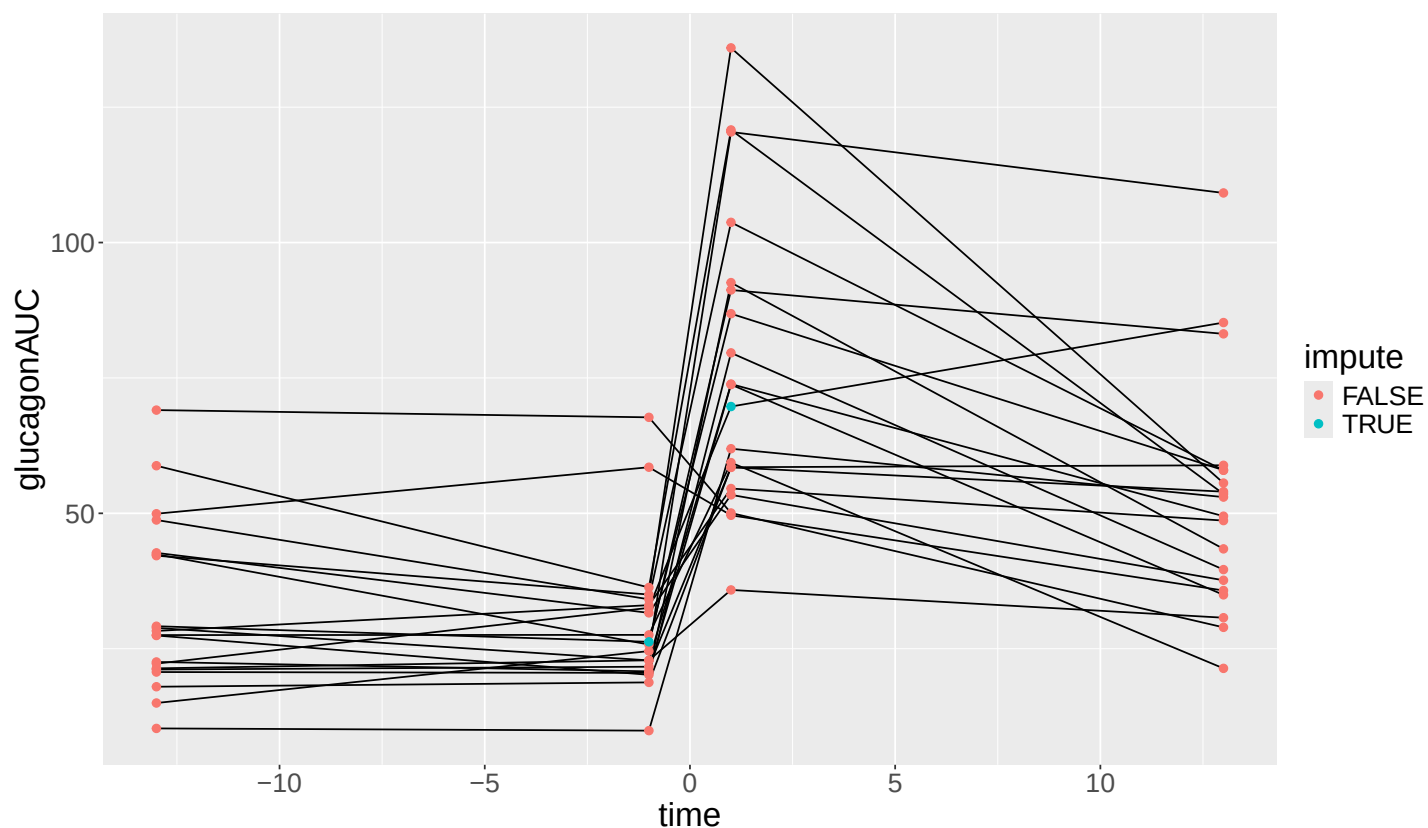
It is possible to extract the most likely value for these missing observations using the fitted function with argument `impute=TRUE`:

```
eData <- fitted(eUN.lmmNA, type = "outcome", keep.data = TRUE)
eData <- eData[order(eData$id,eData$time),]
eData[eData$id %in% eData[eData$impute,"id"],c("id","visit","time","glucagonAUC","impute")]
```

```
id visit time glucagonAUC impute
15 15     1  -13     22.244 FALSE
35 15     2   -1     32.544 FALSE
55 15     3    1     69.719  TRUE
75 15     4   13     85.222 FALSE
5  5      1  -13     29.151 FALSE
25 5      2   -1     26.270  TRUE
45 5      3    1     86.859 FALSE
65 5      4   13     57.970 FALSE
```

Missing outcome values in the dataset have been replaced by its most likely value (which is the same as the dynamic prediction, described previously). A column `impute` has also been added to differentiate between the modeled and observed value. Visually:

```
ggplot(eData, aes(x=time,y=glucagonAUC, group=id)) + geom_line() + geom_point(aes(color=impute
))
```



It is possible to sample from the estimated distribution of the missing value instead of using the most likely value, e.g. accounting for residual variance and uncertainty related to parameter estimation:

```
index.na <- which(is.na(gastricbypassL$glucagonAUC))
set.seed(1)
fitted(eUN.lmmNA, type = "impute", se = c(TRUE,TRUE))[index.na]
set.seed(2)
fitted(eUN.lmmNA, type = "impute", se = c(TRUE,TRUE))[index.na]
set.seed(3)
fitted(eUN.lmmNA, type = "impute", se = c(TRUE,TRUE))[index.na]
```

```
[1] 21.932 75.390
[1] 20.060 75.428
[1] 19.610 60.684
```

8.4 Multiple imputation

The `mlmm` function can be used to perform stratified analyses, typically useful when performing multiple imputations. Consider the wide format of the dataset where a few values are missing:

```
data(gastricbypassW, package = "LMMstar")
colSums(is.na(gastricbypassW))
```

```
      id      weight1      weight2      weight3      weight4 glucagonAUC1 glucagonAUC2
      0           0           0           0           0           0           1
glucagonAUC3 glucagonAUC4
      1           0
```

We use `mice` to generate a number of imputed datasets (here 5):

```
library(mice)
set.seed(10)
gastricbypassW.mice <- mice(gastricbypassW, m = 5, printFlag = FALSE)
gastricbypassW.NNA <- complete(gastricbypassW.mice, action = "long")
table(gastricbypassW.NNA$.imp)
```

Warning message:

Number of logged events: 108

```
 1  2  3  4  5
20 20 20 20 20
```

We can then use `mlmm` to perform a separate linear regression per dataset:

```
e.mlmm <- mlmm(glucagonAUC3~glucagonAUC2+weight2, data=gastricbypassW.NNA,
               by = ".imp", effects = "weight2=0", trace = FALSE)
model.tables(e.mlmm)
```

```
by parameter estimate      se      df      lower      upper      p.value
1  1  weight2 -0.79725 0.31970 17.003 -1.4717 -0.12276 0.0232403
2  2  weight2 -0.83352 0.29798 17.003 -1.4622 -0.20486 0.0123742
3  3  weight2 -0.90658 0.28187 17.003 -1.5013 -0.31189 0.0050657
4  4  weight2 -0.90648 0.28183 17.003 -1.5011 -0.31189 0.0050638
5  5  weight2 -0.82477 0.30247 17.003 -1.4629 -0.18663 0.0143469
```

and pool the results using Rubin's rule:

```
model.tables(e.mlmm, method = "pool.rubin")
```

```
      estimate      se      df      lower      upper      p.value
<1, 5> -0.85372 0.30212 14.741 -1.4987 -0.20878 0.012949
```

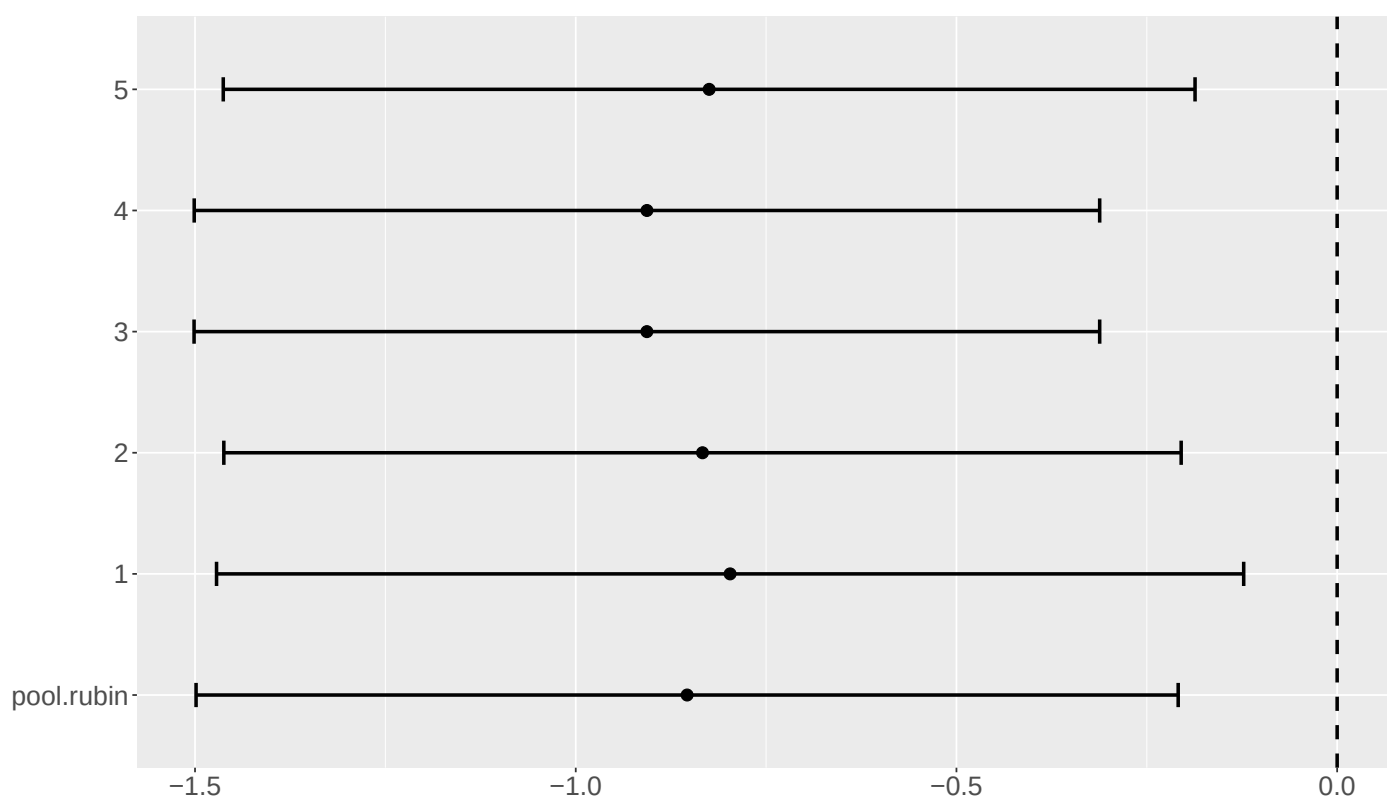
This matches (almost exactly, only the degrees of freedom are a little different) the results obtained with the mice package:

```
e.mice <- with(data=gastricbypassW.mice,exp=lm(glucagonAUC3~glucagonAUC2+weight2))
summary(pool(e.mice))
```

	term	estimate	std.error	statistic	df	p.value
1	(Intercept)	178.988359	36.52589	4.900314	14.703	0.00020353
2	glucagonAUC2	0.027599	0.41848	0.065951	15.132	0.94828055
3	weight2	-0.853721	0.30212	-2.825775	14.737	0.01295073

One can use the plot function to obtain a forest plot of the individual estimates along with the pooled estimate:

```
plot(e.mlmm, method = c("pool.rubin", "none"))
```



9 Data generation

Simulate some data in the wide format:

```
set.seed(10) ## ensure reproductibility
n.obs <- 100
n.times <- 4
mu <- rep(0,4)
gamma <- matrix(0, nrow = n.times, ncol = 10) ## add interaction
gamma[,6] <- c(0,1,1.5,1.5)
dW <- sampleRem(n.obs, n.times = n.times, mu = mu, gamma = gamma, format = "wide")
head(round(dW,3))
```

	id	X1	X2	X3	X4	X5	X6	X7	X8	X9	X10	Y1	Y2	Y3	Y4
1	1	1	0	1	1	0	-0.367	1.534	-1.894	1.729	0.959	1.791	2.429	3.958	2.991
2	2	1	0	1	2	0	-0.410	2.065	1.766	0.761	-0.563	2.500	4.272	3.002	2.019
3	3	0	0	2	1	0	-1.720	-0.178	2.357	1.966	1.215	-3.208	-5.908	-4.277	-5.154
4	4	0	0	0	1	0	0.923	-2.089	0.233	1.307	-0.906	-2.062	0.397	1.757	-1.380
5	5	0	0	2	1	0	0.987	5.880	0.385	0.028	0.820	7.963	7.870	7.388	8.609
6	6	0	0	1	1	2	-1.075	0.479	2.202	0.900	-0.739	0.109	-1.602	-1.496	-1.841

Simulate some data in the long format:

```
set.seed(10) ## ensure reproductibility
dL <- sampleRem(n.obs, n.times = n.times, mu = mu, gamma = gamma, format = "long")
head(dL)
```

	id	visit	Y	X1	X2	X3	X4	X5	X6	X7	X8	X9	X10
1	1	1	1.7914	1	0	1	1	0	-0.36653	1.5338	-1.8944	1.72887	0.95925
2	1	2	2.4286	1	0	1	1	0	-0.36653	1.5338	-1.8944	1.72887	0.95925
3	1	3	3.9583	1	0	1	1	0	-0.36653	1.5338	-1.8944	1.72887	0.95925
4	1	4	2.9912	1	0	1	1	0	-0.36653	1.5338	-1.8944	1.72887	0.95925
5	2	1	2.5002	1	0	1	2	0	-0.40975	2.0654	1.7658	0.76133	-0.56302
6	2	2	4.2724	1	0	1	2	0	-0.40975	2.0654	1.7658	0.76133	-0.56302

10 Modifying default options

The `LMMstar.options` method enable to get and set the default options used by the package. For instance, the default option for the information matrix is:

```
LMMstar.options("type.information")
```

```
$type.information  
[1] "observed"
```

To change the default option to "expected" (faster to compute but less accurate p-values and confidence intervals in small samples) use:

```
LMMstar.options(type.information = "expected")
```

To restore the original default options do:

```
LMMstar.options(reinitialise = TRUE)
```

11 R session

Details of the R session used to generate this document:

```
sessionInfo()
```

R version 4.3.3 (2024-02-29)

Platform: x86_64-pc-linux-gnu (64-bit)

Running under: Ubuntu 22.04.4 LTS

Matrix products: default

BLAS: /usr/lib/x86_64-linux-gnu/blas/libblas.so.3.10.0

LAPACK: /usr/lib/x86_64-linux-gnu/lapack/liblapack.so.3.10.0

locale:

[1] LC_CTYPE=en_US.UTF-8	LC_NUMERIC=C	LC_TIME=en_US.UTF-8
[4] LC_COLLATE=en_US.UTF-8	LC_MONETARY=en_US.UTF-8	LC_MESSAGES=en_US.UTF-8
[7] LC_PAPER=en_US.UTF-8	LC_NAME=C	LC_ADDRESS=C
[10] LC_TELEPHONE=C	LC_MEASUREMENT=en_US.UTF-8	LC_IDENTIFICATION=C

time zone: Europe/Copenhagen

tzcode source: system (glibc)

attached base packages:

[1] grid	parallel	stats	graphics	grDevices	utils	datasets	methods	base
----------	----------	-------	----------	-----------	-------	----------	---------	------

other attached packages:

[1] mice_3.16.0	emmeans_1.10.0	rlang_1.1.3	numDeriv_2016.8-1.1
[5] doParallel_1.0.17	iterators_1.0.14	foreach_1.5.2	copula_1.1-3
[9] multcomp_1.4-25	TH.data_1.1-2	MASS_7.3-60.0.1	survival_3.5-8
[13] mvtnorm_1.2-4	lme4_1.1-35.2	Matrix_1.6-5	lava_1.8.0
[17] nlme_3.1-163	LMMstar_1.1.0	ggpubr_0.6.0	ggplot2_3.5.1

loaded via a namespace (and not attached):

[1] pbapply_1.7-2	gridExtra_2.3	pspline_1.0-19	remotes_2.5.0
[5] sandwich_3.1-0	magrittr_2.0.3	butils.base_1.3	compiler_4.3.3
[9] mgcv_1.9-1	systemfonts_1.0.6	vctr_0.6.5	gsl_2.1-8
[13] stringr_1.5.1	profvis_0.3.8	shape_1.4.6.1	pkgconfig_2.0.3
[17] fastmap_1.1.1	backports_1.4.1	ellipsis_0.3.2	labeling_0.4.3
[21] utf8_1.2.4	promises_1.2.1	qqtest_1.2.0	sessioninfo_1.2.2
[25] nloptr_2.0.3	ragg_1.3.0	purrr_1.0.2	jomo_2.7-6
[29] glmnet_4.1-8	cachem_1.0.8	later_1.3.2	pan_1.9
[33] broom_1.0.5	R6_2.5.1	stringi_1.8.3	rpart_4.1.23
[37] parallelly_1.37.1	car_3.1-2	boot_1.3-30	pkgload_1.3.4
[41] estimability_1.5	Rcpp_1.0.12	future.apply_1.11.2	zoo_1.8-12
[45] usethis_2.2.3	nnet_7.3-19	httpuv_1.6.15	splines_4.3.3
[49] tidyselect_1.2.1	abind_1.4-5	codetools_0.2-19	miniUI_0.1.1.1
[53] listenv_0.9.1	pkgbuild_1.4.4	lattice_0.22-5	tibble_3.2.1

[57] shiny_1.8.1.1	withr_3.0.0	coda_0.19-4.1	future_1.33.2
[61] urlchecker_1.0.1	pillar_1.9.0	carData_3.0-5	stats4_4.3.3
[65] pcaPP_2.0-4	generics_0.1.3	munsell_0.5.1	scales_1.3.0
[69] minqa_1.2.6	globals_0.16.3	xtable_1.8-4	glue_1.7.0
[73] ADGofTest_0.3	tools_4.3.3	data.table_1.15.4	ggsignif_0.6.4
[77] fs_1.6.3	cowplot_1.1.3	tidyr_1.3.1	devtools_2.4.5
[81] colorspace_2.1-0	cli_3.6.2	textshaping_0.3.7	fansi_1.0.6
[85] dplyr_1.1.4	gtable_0.3.5	rstatix_0.7.2	stabledist_0.7-1
[89] digest_0.6.35	pbkrtest_0.5.2	htmlwidgets_1.6.4	farver_2.1.1
[93] memoise_2.0.1	htmltools_0.5.8.1	lifecycle_1.0.4	mitml_0.4-5
[97] mime_0.12			

References

- Christensen, R. (2011). *Plane answers to complex questions (4th edition)*, volume 35. Springer.
- Geyer, C. J. (2013). Asymptotics of maximum likelihood without the lln or clt or sample size going to infinity. In *Advances in Modern Statistical Theory and Applications: A Festschrift in honor of Morris L. Eaton*, volume 10, pages 1–25. Institute of Mathematical Statistics.
- Lakens, D. (2013). Calculating and reporting effect sizes to facilitate cumulative science: a practical primer for t-tests and anovas. *Frontiers in psychology*, 4:863.
- Lipsitz, S. R., Fitzmaurice, G. M., Ibrahim, J. G., Gelber, R., and Lipshultz, S. (2002). Parameter estimation in longitudinal studies with outcome-dependent follow-up. *Biometrics*, 58(3):621–630.
- Oldford, R. W. (2016). Self-calibrating quantile–quantile plots. *The American Statistician*, 70(1):74–90.
- Ozenne, B., Nørgaard, M., Pernet, C., and Ganz, M. (2025). A sensitivity analysis of preprocessing pipelines: Toward a solution for multiverse analyses. *Imaging Neuroscience*, 3:imag_a__00523.
- Pipper, C. B., Ritz, C., and Bisgaard, H. (2012). A versatile method for confirmatory evaluation of the effects of a covariate in multiple models. *Journal of the Royal Statistical Society: Series C (Applied Statistics)*, 61(2):315–326.
- Thomadakis, C., Pantazis, N., and Touloumi, G. (2023). Issues with the expected information matrix of linear mixed models provided by popular statistical packages under missingness at random dropout. *Statistics in Medicine*, 42(16):2873–2885.

Appendix A S3 methods

```
print(M.class2method, quote = FALSE)
```

	lmm	Wald_lmm	rbindWald_lmm	mlmm	effect_lmm	confint_lmm
anova	x			x		
autoplot	x	x		x		
coef	x	x	x	x		
confint	x	x	x	x	x	
df.residual	x			x		
dummy.coef	x					
effects	x					
estimate	x			x		
fitted	x			x		
formula	x					
iid	x	x	x	x		
influence	x	x	x	x		
information	x					
levels	x					
lmm		x	x			
logLik	x			x		
model.frame	x					
model.matrix	x					
model.tables	x	x	x	x	x	
moments	x					
nobs	x			x		
partialCor	x					
plot	x	x		x		
pool		x	x			
predict	x			x		
print	x	x		x	x	x
profile	x					
ranef	x			x		
rbind		x	x			
resample	x			x		
residuals	x			x		
score	x			x		
sigma	x					
summary	x	x		x	x	
terms	x					
variable.names	x			x		
vcov	x	x	x	x		
weights		x				

Appendix B Likelihood and derivatives in a linear mixed model

In `lmm`:

- the argument `formula` specifies $\mu(\boldsymbol{\Theta}, \mathbf{X})$ the modeled mean of the outcome $\mathbf{Y}$ conditional a vector of p covariates $\mathbf{X}$.
- the argument `repetition` (typically equal to `~time|id`) enable to interpret $\mathbf{Y}$ as a vector of m outcomes (indexed by `time`) for which we observe independent replicates (across `id`).
- the argument `structure` define a model for the residual variance-covariance $\Omega(\boldsymbol{\Theta}, \mathbf{X})$.
- the right-hand side of argument `weights` defined a vector of weights $\omega = (w_1, \dots, w_n)$. The left hand side defines the number of replicates $\mathbf{r} = (r_1, \dots, r_T)$ which were averaged to obtain the outcome $\mathbf{Y}$. When left un-specified, the weights and number of replicates are considered equal to 1.

B.1 Log-likelihood

With n independent replicates $(\mathbf{Y}_1, \dots, \mathbf{Y}_n)$ and $(\mathbf{X}_1, \dots, \mathbf{X}_n)$, the restricted Gaussian log-likelihood is:

$$\begin{aligned} \mathcal{L}(\boldsymbol{\Theta} | \mathbf{Y}, \mathbf{X}) = & \frac{p}{2} \log(2\pi) - \frac{1}{2} \log \left(\left| \sum_{i=1}^n w_i \mathbf{X}_i \Omega_i^{-1}(\boldsymbol{\Theta}) \mathbf{X}_i^\top \right| \right) \\ & + \sum_{i=1}^n w_i \left(-\frac{m}{2} \log(2\pi) - \frac{1}{2} \log |\Omega_i(\boldsymbol{\Theta})| - \frac{1}{2} \boldsymbol{\varepsilon}_i(\boldsymbol{\Theta}) \Omega_i(\boldsymbol{\Theta})^{-1} \boldsymbol{\varepsilon}_i(\boldsymbol{\Theta})^\top \right) \end{aligned} \quad (\text{A})$$

where $\boldsymbol{\varepsilon}_i(\boldsymbol{\Theta}) = \mathbf{Y}_i - \mu(\boldsymbol{\Theta}, \mathbf{X}_i)$ denotes the residuals. This is what the `logLik` method is computing for the REML criteria. The red term is specific to the REML criteria and prevents from computing individual contributions to the likelihood⁴. The blue term is what `logLik` outputs for the ML criteria when setting the argument `indiv` to `TRUE`.

Note: when having technical replicates $\mathbf{Y}^*$ and averaging over those to obtain $\mathbf{Y}$ rescales the residual variance-covariance matrix. Denote by $\Omega_i^*(\boldsymbol{\Theta})$ the 'single replicate' residual variance-covariance matrix which is to be specified in argument `structure`. Assuming independent replicates, i.e. for two replicates r and r' : $\text{Cov}(\mathbf{Y}_{ir}^*, \mathbf{Y}_{ir'}^*) = \Omega_i^*(\boldsymbol{\Theta}) \mathbf{1}_{r=r'}$, we can relate $\Omega_i(\boldsymbol{\Theta})$ to $\Omega_i^*(\boldsymbol{\Theta})$:

$$\Omega_{itt'}(\boldsymbol{\Theta}) = \text{Cov}(Y_{it}, Y_{it'}) = \frac{1}{r_{it} r_{it'}} \sum_{r=1}^{r_{it}} \sum_{r'=1}^{r_{it'}} \text{Cov}(Y_{itr}^*, Y_{it'r'}^*) = \frac{\min(r_{it}, r_{it'})}{r_{it} r_{it'}} \Omega_{itt'}^*(\boldsymbol{\Theta}) = \frac{1}{\max(r_{it}, r_{it'})} \Omega_{itt'}^*(\boldsymbol{\Theta})$$

which reduces to dividing by the number of replicates in the special case where the number of replicates only varies across subjects but is the same across outcomes within a subject. Even in this case having technical replicate is not equivalent to re-weighting the log-likelihood, e.g. the 'replication' weight creates an additional additive for the ML log term (i.e. will cancel out after differentiation) while it rescales the REML and the residual term (i.e. will contribute even after differentiation).

⁴The REML is the likelihood of the observations divided by the prior on the estimated mean parameters $\hat{\boldsymbol{\Theta}}_\mu \sim \mathcal{N}(\mu, (\mathbf{X} \Omega^{-1}(\boldsymbol{\Theta}) \mathbf{X}^\top)^{-1})$. This corresponds to $\frac{1}{\sqrt{2\pi^p} |(\sum_{i=1}^n \mathbf{X}_i \Omega_i^{-1}(\boldsymbol{\Theta}) \mathbf{X}_i^\top)^{-1}|} \exp \left(-(\hat{\boldsymbol{\Theta}}_\mu - \mu) (2 \sum_{i=1}^n \mathbf{X}_i \Omega_i^{-1}(\boldsymbol{\Theta}) \mathbf{X}_i^\top)^{-1} (\hat{\boldsymbol{\Theta}}_\mu - \mu)^\top \right)$. Since μ will be estimated to be $\boldsymbol{\Theta}_\mu$, the exponential term equals 1 and thus does not contribute to the log-likelihood. One divided by the other term gives $\sqrt{2\pi^p} |(\sum_{i=1}^n \mathbf{X}_i \Omega_i^{-1}(\boldsymbol{\Theta}) \mathbf{X}_i^\top)|^{-1}$. The log of this term equals the red term.

B.2 Score

Using that $\partial \log(\det(X)) = \text{tr}(X^{-1}\partial(X))$, the score is obtained by derivating once the log-likelihood, i.e., for $\theta \in \Theta$:

$$\begin{aligned} \mathcal{S}(\theta) &= \frac{\partial \mathcal{L}(\Theta|\mathbf{Y}, \mathbf{X})}{\partial \theta} = \frac{1}{2} \text{tr} \left(\left(\sum_{j=1}^n w_j \mathbf{X}_j \Omega_j^{-1}(\Theta) \mathbf{X}_j^\top \right)^{-1} \left(\sum_{i=1}^n w_i \mathbf{X}_i \Omega_i^{-1}(\Theta) \frac{\partial \Omega_i(\Theta)}{\partial \theta} \Omega_i(\Theta)^{-1} \mathbf{X}_i^\top \right) \right) \\ &+ \sum_{i=1}^n w_i \left(-\frac{1}{2} \text{tr} \left(\Omega_i(\Theta)^{-1} \frac{\partial \Omega_i(\Theta)}{\partial \theta} \right) + \frac{\partial \mu(\Theta, \mathbf{X}_i)}{\partial \theta} \Omega_i(\Theta)^{-1} \boldsymbol{\varepsilon}_i(\Theta)^\top + \frac{1}{2} \boldsymbol{\varepsilon}_i(\Theta) \Omega_i(\Theta)^{-1} \frac{\partial \Omega_i(\Theta)}{\partial \theta} \Omega_i(\Theta)^{-1} \boldsymbol{\varepsilon}_i(\Theta)^\top \right). \end{aligned}$$

This is what the **score** method is computing for the REML criteria (blue and red) and the ML (blue only) criteria. When the individual contribution is requested (**indiv=TRUE**), **score** output:

$$\begin{aligned} \mathcal{S}_i(\theta) &= \frac{1}{2} \text{tr} \left(\left(\sum_{j=1}^n w_j \mathbf{X}_j \Omega_j^{-1}(\Theta) \mathbf{X}_j^\top \right)^{-1} \left(w_i \mathbf{X}_i \Omega_i^{-1}(\Theta) \frac{\partial \Omega_i(\Theta)}{\partial \theta} \Omega_i(\Theta)^{-1} \mathbf{X}_i^\top \right) \right) \\ &+ w_i \left(-\frac{1}{2} \text{tr} \left(\Omega_i(\Theta)^{-1} \frac{\partial \Omega_i(\Theta)}{\partial \theta} \right) + \frac{\partial \mu(\Theta, \mathbf{X}_i)}{\partial \theta} \Omega_i(\Theta)^{-1} \boldsymbol{\varepsilon}_i(\Theta)^\top + \frac{1}{2} \boldsymbol{\varepsilon}_i(\Theta) \Omega_i(\Theta)^{-1} \frac{\partial \Omega_i(\Theta)}{\partial \theta} \Omega_i(\Theta)^{-1} \boldsymbol{\varepsilon}_i(\Theta)^\top \right). \end{aligned}$$

B.3 Hessian

Derivating a second time the log-likelihood gives the hessian, $\mathcal{H}(\Theta)$, with element:

$$\begin{aligned} \mathcal{H}(\theta, \theta') &= \frac{\partial^2 \mathcal{L}(\Theta|\mathbf{Y}, \mathbf{X})}{\partial \theta \partial \theta'} = \frac{\partial \mathcal{S}(\theta)}{\partial \theta'} \\ &= \frac{1}{2} \text{tr} \left(\left(\sum_{j=1}^n w_j \mathbf{X}_j \Omega_j^{-1}(\Theta) \mathbf{X}_j^\top \right)^{-1} \left\{ \sum_{i=1}^n w_i \mathbf{X}_i \Omega_i^{-1}(\Theta) \left(\frac{\partial^2 \Omega_i(\Theta)}{\partial \theta \partial \theta'} - 2 \frac{\partial \Omega_i(\Theta)}{\partial \theta} \Omega_i^{-1}(\Theta) \frac{\partial \Omega_i(\Theta)}{\partial \theta'} \right) \Omega_i(\Theta)^{-1} \mathbf{X}_i^\top \right. \right. \\ &\quad \left. \left. + \left(\sum_{i=1}^n w_i \mathbf{X}_i \Omega_i^{-1}(\Theta) \frac{\partial \Omega_i(\Theta)}{\partial \theta} \Omega_i(\Theta)^{-1} \mathbf{X}_i^\top \right) \left(\sum_{j=1}^n w_j \mathbf{X}_j \Omega_j^{-1}(\Theta) \mathbf{X}_j^\top \right)^{-1} \left(\sum_{i=1}^n w_i \mathbf{X}_i \Omega_i^{-1}(\Theta) \frac{\partial \Omega_i(\Theta)}{\partial \theta'} \Omega_i(\Theta)^{-1} \mathbf{X}_i^\top \right) \right\} \right) \\ &\quad + \sum_{i=1}^n w_i \left(\frac{1}{2} \text{tr} \left(\Omega_i(\Theta)^{-1} \frac{\partial \Omega_i(\Theta)}{\partial \theta'} \Omega_i(\Theta)^{-1} \frac{\partial \Omega_i(\Theta)}{\partial \theta} - \Omega_i(\Theta)^{-1} \frac{\partial^2 \Omega_i(\Theta)}{\partial \theta \partial \theta'} \right) \right. \\ &\quad \left. - \frac{\partial \mu(\Theta, \mathbf{X}_i)}{\partial \theta} \Omega_i(\Theta)^{-1} \frac{\partial \Omega_i(\Theta)}{\partial \theta'} \Omega_i(\Theta)^{-1} \boldsymbol{\varepsilon}_i(\Theta)^\top - \frac{\partial \mu(\Theta, \mathbf{X}_i)}{\partial \theta} \Omega_i(\Theta)^{-1} \frac{\partial \mu(\Theta, \mathbf{X}_i)}{\partial \theta'} \right. \\ &\quad \left. + \frac{1}{2} \boldsymbol{\varepsilon}_i(\Theta) \Omega_i(\Theta)^{-1} \left(\frac{\partial^2 \Omega_i(\Theta)}{\partial \theta \partial \theta'} - \frac{\partial \Omega_i(\Theta)}{\partial \theta'} \Omega_i(\Theta)^{-1} \frac{\partial \Omega_i(\Theta)}{\partial \theta} - \frac{\partial \Omega_i(\Theta)}{\partial \theta} \Omega_i(\Theta)^{-1} \frac{\partial \Omega_i(\Theta)}{\partial \theta'} \right) \Omega_i(\Theta)^{-1} \boldsymbol{\varepsilon}_i(\Theta)^\top \right). \end{aligned}$$

where (θ, θ') are either two mean parameters, or two variance parameters, or θ is a mean parameter and θ' as variance parameter. The case θ variance parameter and θ mean parameter can be deduced by symmetry; doing so enable some simplifications since $\frac{\partial \mu(\Theta, \mathbf{X}_i)}{\partial \theta'} \frac{\partial \Omega(\Theta, \mathbf{X}_i)}{\partial \theta} = 0$.

The **information** method will (by default) return the (observed) information which is the opposite of the hessian. So the previous formula shows **-1** times what the **information** method is computing for the REML criteria (blue and red) and the ML (blue only) criteria, with the default option **type.information="observed"**. When the individual contribution is requested (**indiv=TRUE**), **information**

output:

$$\begin{aligned}
\mathcal{H}_i(\theta, \theta') &= \frac{1}{2} \text{tr} \left(\left(\sum_{j=1}^n w_j \mathbf{X}_j \Omega_j^{-1}(\boldsymbol{\Theta}) \mathbf{X}_j^\top \right)^{-1} \left\{ w_i \mathbf{X}_i \Omega_i^{-1}(\boldsymbol{\Theta}) \left(\frac{\partial^2 \Omega_i(\boldsymbol{\Theta})}{\partial \theta \partial \theta'} - 2 \frac{\partial \Omega_i(\boldsymbol{\Theta})}{\partial \theta} \Omega_i^{-1}(\boldsymbol{\Theta}) \frac{\partial \Omega_i(\boldsymbol{\Theta})}{\partial \theta'} \right) \Omega_i(\boldsymbol{\Theta})^{-1} \mathbf{X}_i^\top \right. \right. \\
&+ 0.5 \left(w_i \mathbf{X}_i \Omega_i^{-1}(\boldsymbol{\Theta}) \frac{\partial \Omega_i(\boldsymbol{\Theta})}{\partial \theta} \Omega_i(\boldsymbol{\Theta})^{-1} \mathbf{X}_i^\top \right) \left(\sum_{j=1}^n w_j \mathbf{X}_j \Omega_j^{-1}(\boldsymbol{\Theta}) \mathbf{X}_j^\top \right)^{-1} \left(\sum_{j=1}^n w_j \mathbf{X}_j \Omega_j^{-1}(\boldsymbol{\Theta}) \frac{\partial \Omega_j(\boldsymbol{\Theta})}{\partial \theta'} \Omega_j(\boldsymbol{\Theta})^{-1} \mathbf{X}_j^\top \right) \left. \right\} \\
&+ 0.5 \left(\sum_{j=1}^n w_j \mathbf{X}_j \Omega_j^{-1}(\boldsymbol{\Theta}) \frac{\partial \Omega_j(\boldsymbol{\Theta})}{\partial \theta} \Omega_j(\boldsymbol{\Theta})^{-1} \mathbf{X}_j^\top \right) \left(\sum_{j=1}^n w_j \mathbf{X}_j \Omega_j^{-1}(\boldsymbol{\Theta}) \mathbf{X}_j^\top \right)^{-1} \left(w_i \mathbf{X}_i \Omega_i^{-1}(\boldsymbol{\Theta}) \frac{\partial \Omega_i(\boldsymbol{\Theta})}{\partial \theta'} \Omega_i(\boldsymbol{\Theta})^{-1} \mathbf{X}_i^\top \right) \left. \right\} \\
&+ w_i \left(\frac{1}{2} \text{tr} \left(\Omega_i(\boldsymbol{\Theta})^{-1} \frac{\partial \Omega_i(\boldsymbol{\Theta})}{\partial \theta'} \Omega_i(\boldsymbol{\Theta})^{-1} \frac{\partial \Omega_i(\boldsymbol{\Theta})}{\partial \theta} - \Omega_i(\boldsymbol{\Theta})^{-1} \frac{\partial^2 \Omega_i(\boldsymbol{\Theta})}{\partial \theta \partial \theta'} \right) \right. \\
&\quad - \frac{\partial \mu(\boldsymbol{\Theta}, \mathbf{X}_i)}{\partial \theta} \Omega_i(\boldsymbol{\Theta})^{-1} \frac{\partial \Omega_i(\boldsymbol{\Theta})}{\partial \theta'} \Omega_i(\boldsymbol{\Theta})^{-1} \boldsymbol{\varepsilon}_i(\boldsymbol{\Theta})^\top - \frac{\partial \mu(\boldsymbol{\Theta}, \mathbf{X}_i)}{\partial \theta} \Omega_i(\boldsymbol{\Theta})^{-1} \frac{\partial \mu(\boldsymbol{\Theta}, \mathbf{X}_i)^\top}{\partial \theta'} \\
&\quad \left. + \frac{1}{2} \boldsymbol{\varepsilon}_i(\boldsymbol{\Theta}) \Omega_i(\boldsymbol{\Theta})^{-1} \left(\frac{\partial^2 \Omega_i(\boldsymbol{\Theta})}{\partial \theta \partial \theta'} - \frac{\partial \Omega_i(\boldsymbol{\Theta})}{\partial \theta'} \Omega_i(\boldsymbol{\Theta})^{-1} \frac{\partial \Omega_i(\boldsymbol{\Theta})}{\partial \theta} - \frac{\partial \Omega_i(\boldsymbol{\Theta})}{\partial \theta} \Omega_i(\boldsymbol{\Theta})^{-1} \frac{\partial \Omega_i(\boldsymbol{\Theta})}{\partial \theta'} \right) \Omega_i(\boldsymbol{\Theta})^{-1} \boldsymbol{\varepsilon}_i(\boldsymbol{\Theta})^\top \right).
\end{aligned}$$

B.4 Expected Hessian

A possible simplification is to use the expected Hessian at the Maximum (Restricted) Likelihood. Indeed for any deterministic matrix A :

- $\mathbb{E}[A(\mathbf{Y}_i - \mu(\boldsymbol{\Theta}, \mathbf{X}_i))^\top | \mathbf{X}_i] = 0$
- $\mathbb{E}[(\mathbf{Y}_i - \mu(\boldsymbol{\Theta}, \mathbf{X}_i))A(\mathbf{Y}_i - \mu(\boldsymbol{\Theta}, \mathbf{X}_i))^\top | \mathbf{X}_i] = \text{tr}(A \text{Var}(\mathbf{Y}_i - \mu(\boldsymbol{\Theta}, \mathbf{X}_i)))$

when $\mathbb{E}[\mathbf{Y}_i - \mu(\boldsymbol{\Theta}, \mathbf{X}_i)] = 0$. This leads to:

$$\begin{aligned}
&\mathbb{E}[\mathcal{H}(\theta, \theta') | \mathbf{X}] \\
&= \frac{1}{2} \text{tr} \left(\left(\sum_{i=1}^n w_i \mathbf{X}_i \Omega_i^{-1}(\boldsymbol{\Theta}) \mathbf{X}_i^\top \right)^{-1} \left\{ \sum_{i=1}^n w_i \mathbf{X}_i \Omega_i^{-1}(\boldsymbol{\Theta}) \left(\frac{\partial^2 \Omega_i(\boldsymbol{\Theta})}{\partial \theta \partial \theta'} - 2 \frac{\partial \Omega_i(\boldsymbol{\Theta})}{\partial \theta} \Omega_i^{-1}(\boldsymbol{\Theta}) \frac{\partial \Omega_i(\boldsymbol{\Theta})}{\partial \theta'} \right) \Omega_i(\boldsymbol{\Theta})^{-1} \mathbf{X}_i^\top \right. \right. \\
&+ \left(\sum_{i=1}^n w_i \mathbf{X}_i \Omega_i^{-1}(\boldsymbol{\Theta}) \frac{\partial \Omega_i(\boldsymbol{\Theta})}{\partial \theta} \Omega_i(\boldsymbol{\Theta})^{-1} \mathbf{X}_i^\top \right) \left(\sum_{i=1}^n w_i \mathbf{X}_i \Omega_i^{-1}(\boldsymbol{\Theta}) \mathbf{X}_i^\top \right)^{-1} \left(\sum_{i=1}^n w_i \mathbf{X}_i \Omega_i^{-1}(\boldsymbol{\Theta}) \frac{\partial \Omega_i(\boldsymbol{\Theta})}{\partial \theta'} \Omega_i(\boldsymbol{\Theta})^{-1} \mathbf{X}_i^\top \right) \left. \right\} \\
&+ \sum_{i=1}^n w_i \left(-\frac{1}{2} \text{tr} \left(\Omega_i(\boldsymbol{\Theta})^{-1} \frac{\partial \Omega_i(\boldsymbol{\Theta})}{\partial \theta'} \Omega_i(\boldsymbol{\Theta})^{-1} \frac{\partial \Omega_i(\boldsymbol{\Theta})}{\partial \theta} \right) - \frac{\partial \mu(\boldsymbol{\Theta}, \mathbf{X}_i)}{\partial \theta} \Omega_i(\boldsymbol{\Theta})^{-1} \frac{\partial \mu(\boldsymbol{\Theta}, \mathbf{X}_i)^\top}{\partial \theta'} \right) \quad (\text{B})
\end{aligned}$$

which is what `information` output when the argument `type.information` is set to "expected" (up to a factor **-1**). A similar simplification is used when `indiv` is set to `TRUE`.

B.5 Degrees of freedom

Degrees of freedom are computed using a Satterthwaite approximation, i.e. for an estimate coefficient $\hat{\beta} \in \hat{\boldsymbol{\Theta}}$ with standard error $\sigma_{\hat{\beta}}$, the degree of freedom is:

$$df(\sigma_{\hat{\beta}}) = \frac{2\sigma_{\hat{\beta}}^4}{\text{Var}[\hat{\sigma}_{\hat{\beta}}]}$$

Using a first order Taylor expansion we can approximate the variance term as:

$$\begin{aligned}\mathbb{V}ar\left[\hat{\sigma}_{\hat{\beta}}\right] &\approx \frac{\partial \hat{\sigma}_{\hat{\beta}}}{\partial \Theta} \Sigma_{\Theta} \frac{\partial \hat{\sigma}_{\hat{\beta}}}{\partial \Theta}^{\top} \\ &\approx c_{\beta} \left(\hat{\mathcal{I}}_{\Theta}\right)^{-1} \frac{\partial \hat{\mathcal{I}}_{\Theta}}{\partial \Theta} \left(\hat{\mathcal{I}}_{\Theta}\right)^{-1} c_{\beta}^{\top} \Sigma_{\Theta} c_{\beta} \left(\hat{\mathcal{I}}_{\Theta}\right)^{-1} \frac{\partial \hat{\mathcal{I}}_{\Theta}}{\partial \Theta}^{\top} \left(\hat{\mathcal{I}}_{\Theta}\right)^{-1} c_{\beta}\end{aligned}$$

where Σ_{Θ} is the variance-covariance matrix of all model coefficients, $\mathcal{I}_{\Theta}$ the information matrix for all model coefficients, c_{β} a matrix used to select the element relative to β in the first derivative of the information matrix, and $\frac{\partial}{\partial \Theta}$ denotes the vector of derivatives with respect to all model coefficients.

For the derivative of the information (i.e. $-\mathbf{1}$ times the derivative of the hessian) we consider $(\theta, \theta', \theta'')$ are either three mean parameters, or three variance parameters, or θ is a mean parameter and θ' and θ'' are variance parameters, or θ and θ' are mean parameters and θ'' is a variance parameter. The other cases follow by symmetry; doing so enable some simplifications since $\frac{\partial \mu(\Theta, \mathbf{X}_i)}{\partial \theta'} \frac{\partial \Omega(\Theta, \mathbf{X}_i)}{\partial \theta'} \frac{\partial \mu(\Theta, \mathbf{X}_i)}{\partial \theta''} = 0$. We first focus on the derivative of the ML term:

$$\begin{aligned}\frac{\partial \mathcal{H}(\theta, \theta')}{\partial \theta''} &= \sum_{i=1}^n w_i \left(\frac{1}{2} \text{tr} \left(\Omega_i(\Theta)^{-1} \frac{\partial \Omega_i(\Theta)}{\partial \theta''} \Omega_i(\Theta)^{-1} \frac{\partial^2 \Omega_i(\Theta)}{\partial \theta \partial \theta'} - \Omega_i(\Theta)^{-1} \frac{\partial^3 \Omega_i(\Theta)}{\partial \theta \partial \theta' \partial \theta''} - \Upsilon_{\theta \theta' \theta''} - \Upsilon_{\theta' \theta \theta''} \right) \right. \\ &\quad + \frac{\partial \mu(\Theta, \mathbf{X}_i)}{\partial \theta} \Omega_i(\Theta)^{-1} \left(\frac{\partial \Omega_i(\Theta)}{\partial \theta''} \Omega_i(\Theta)^{-1} \frac{\partial \Omega_i(\Theta)}{\partial \theta'} - \frac{\partial^2 \Omega_i(\Theta)}{\partial \theta' \partial \theta''} + \frac{\partial \Omega_i(\Theta)}{\partial \theta'} \Omega_i(\Theta)^{-1} \frac{\partial \Omega_i(\Theta)}{\partial \theta''} \right) \Omega_i(\Theta)^{-1} \boldsymbol{\varepsilon}_i(\Theta)^{\top} \\ &\quad + \frac{\partial \mu(\Theta, \mathbf{X}_i)}{\partial \theta} \Omega_i(\Theta)^{-1} \frac{\partial \Omega_i(\Theta)}{\partial \theta''} \Omega_i(\Theta)^{-1} \frac{\partial \mu(\Theta, \mathbf{X}_i)}{\partial \theta'}^{\top} \\ &\quad \left. + \frac{1}{2} \boldsymbol{\varepsilon}_i(\Theta) \left(\Omega_i(\Theta)^{-1} \frac{\partial^3 \Omega_i(\Theta)}{\partial \theta \partial \theta' \partial \theta''} + \Upsilon_{\theta \theta' \theta''} + \Upsilon_{\theta \theta'' \theta'} + \Upsilon_{\theta' \theta \theta''} + \Upsilon_{\theta' \theta'' \theta} + \Upsilon_{\theta'' \theta \theta'} + \Upsilon_{\theta'' \theta' \theta} \right) \Omega_i(\Theta)^{-1} \boldsymbol{\varepsilon}_i(\Theta)^{\top} \right).\end{aligned}$$

where

$$\begin{aligned}\Upsilon_{\theta \theta' \theta''} &= \Omega_i(\Theta)^{-1} \frac{\partial \Omega_i(\Theta)}{\partial \theta} \Omega_i(\Theta)^{-1} \left(\frac{\partial \Omega_i(\Theta)}{\partial \theta'} \Omega_i(\Theta)^{-1} \frac{\partial \Omega_i(\Theta)}{\partial \theta''} - \frac{\partial^2 \Omega_i(\Theta)}{\partial \theta' \partial \theta''} \right) \\ \Upsilon_{\theta' \theta \theta''} &= \Omega_i(\Theta)^{-1} \frac{\partial \Omega_i(\Theta)}{\partial \theta'} \Omega_i(\Theta)^{-1} \left(\frac{\partial \Omega_i(\Theta)}{\partial \theta} \Omega_i(\Theta)^{-1} \frac{\partial \Omega_i(\Theta)}{\partial \theta''} - \frac{\partial^2 \Omega_i(\Theta)}{\partial \theta \partial \theta''} \right) \\ \Upsilon_{\theta'' \theta \theta'} &= \Omega_i(\Theta)^{-1} \frac{\partial \Omega_i(\Theta)}{\partial \theta''} \Omega_i(\Theta)^{-1} \left(\frac{\partial \Omega_i(\Theta)}{\partial \theta} \Omega_i(\Theta)^{-1} \frac{\partial \Omega_i(\Theta)}{\partial \theta'} - \frac{\partial^2 \Omega_i(\Theta)}{\partial \theta \partial \theta'} \right)\end{aligned}$$

In the special case of the expected information, the derivative of the ML term simplifies to:

$$\frac{\partial \mathbb{E}[\mathcal{H}(\theta, \theta') | \mathbf{X}]}{\partial \theta''} = \sum_{i=1}^n w_i \left(\frac{\partial \mu(\Theta, \mathbf{X}_i)}{\partial \theta} \Omega_i(\Theta)^{-1} \frac{\partial \Omega_i(\Theta)}{\partial \theta''} \Omega_i(\Theta)^{-1} \frac{\partial \mu(\Theta, \mathbf{X}_i)}{\partial \theta'}^{\top} + \frac{1}{2} \text{tr}(\Upsilon_{\theta \theta' \theta''} + \Upsilon_{\theta' \theta \theta''}) \right)$$

The REML term can be re-expressed as:

$$\begin{aligned}\mathcal{H}(\theta, \theta') &= \frac{1}{2} \text{tr} \left(A^{-1} \left(C_{\theta \theta'} + B_{\theta} A^{-1} B_{\theta'} \right) \right) \\ A &= \sum_{i=1}^n w_i \mathbf{X}_i \Omega_i^{-1}(\Theta) \mathbf{X}_i^{\top}\end{aligned}$$

$$\begin{aligned}
B_\theta &= \sum_{i=1}^n w_i \mathbf{X}_i \Omega_i^{-1}(\boldsymbol{\Theta}) \frac{\partial \Omega_i(\boldsymbol{\Theta})}{\partial \theta} \Omega_i(\boldsymbol{\Theta})^{-1} \mathbf{X}_i^\top \\
C_{\theta\theta'} &= \sum_{i=1}^n w_i \mathbf{X}_i \Omega_i^{-1}(\boldsymbol{\Theta}) \left(\frac{\partial^2 \Omega_i(\boldsymbol{\Theta})}{\partial \theta \partial \theta'} - 2 \frac{\partial \Omega_i(\boldsymbol{\Theta})}{\partial \theta} \Omega_i^{-1}(\boldsymbol{\Theta}) \frac{\partial \Omega_i(\boldsymbol{\Theta})}{\partial \theta'} \right) \Omega_i^{-1}(\boldsymbol{\Theta}) \mathbf{X}_i^\top
\end{aligned}$$

Therefore the third derivative of the REML term $\frac{\partial \mathcal{H}(\theta, \theta')}{\partial \theta''}$ equals:

$$\begin{aligned}
&\frac{1}{2} \text{tr} \left(A^{-1} \left(D_{\theta\theta'\theta''} + B_{\theta''} A^{-1} C_{\theta\theta'} + C_{\theta\theta''} A^{-1} B_{\theta'} + B_\theta A^{-1} C_{\theta'\theta''} + B_{\theta''} A^{-1} B_\theta A^{-1} B_{\theta'} + B_\theta A^{-1} B_{\theta''} A^{-1} B_{\theta'} \right) \right) \\
&\text{with } D_{\theta\theta'\theta''} = \sum_{i=1}^n w_i \mathbf{X}_i \left(\Omega_i(\boldsymbol{\Theta})^{-1} \frac{\partial^3 \Omega_i(\boldsymbol{\Theta})}{\partial \theta \partial \theta' \partial \theta''} + \Upsilon_{\theta\theta'\theta''} + \Upsilon_{\theta\theta''\theta'} + \Upsilon_{\theta'\theta\theta''} + \Upsilon_{\theta'\theta''\theta} + \Upsilon_{\theta''\theta\theta'} + \Upsilon_{\theta''\theta'\theta} \right) \Omega_i(\boldsymbol{\Theta})^{-1} \mathbf{X}_i^\top
\end{aligned}$$

Appendix C Likelihood ratio test with the REML criterion

The blue term of Equation A in the log-likelihood is invariant to re-parameterisation while the red term is not. This means that a re-parametrisation of X into $\tilde{X} = BX$ with B invertible would not change the likelihood when using ML but would decrease the log-likelihood by $\log(|B|)$ when using REML.

Let's take an example:

```
## data(dfL, package = "LMMstar")
dfTest <- gastricbypassL[!is.na(gastricbypassL$glucagonAUC),]
dfTest$gluc <- dfTest$glucagonAUC
dfTest$gluc2 <- dfTest$glucagonAUC*2
```

where we multiply one column of the design matrix by 2. As mentioned previously this does not affect the log-likelihood when using ML:

```
eML.UN <- lmm(weight ~ time+gluc, data = dfTest, repetition = ~time|id, method = "ML")
eML.UN2 <- lmm(weight ~ time+gluc, data = dfTest, repetition = ~time|id, method = "ML")
c(logLik(eML.UN), logLik(eML.UN2), logLik(eML.UN) - logLik(eML.UN2))
```

```
[1] -230.62 -230.62 0.00
```

but it does when using REML:

```
eREML.UN <- lmm(weight ~ time + gluc, data = dfTest, repetition = ~time|id, method = "REML")
eREML.UN2 <- lmm(weight ~ time + gluc2, data = dfTest, repetition = ~time|id, method = "REML")
c(logLik(eREML.UN), logLik(eREML.UN2), logLik(eREML.UN) - logLik(eREML.UN2), log(2))
```

```
[1] -235.23462 -235.92777 0.69315 0.69315
```

Therefore, when comparing models with different mean effects there is a risk that the difference (or part of it) in log-likelihood is due to a new parametrisation and not only to a difference in model fit. This would typically be the case when adding an interaction where we can have a smaller restricted log-likelihood when considering a more complex model:

```
set.seed(5)
dfTest$ff <- rbinom(NROW(dfTest), size = 1, prob = 0.5)
logLik(lmm(weight ~ time+gluc, data = dfTest, repetition = ~time|id, method = "REML"))
logLik(lmm(weight ~ time+gluc*ff, data = dfTest, repetition = ~time|id, method = "REML"))
```

```
[1] -235.23
```

```
[1] -238.93
```

This is quite counter-intuitive as more complex model should lead to better fit and would never happen when using ML:

```
logLik(lmm(weight ~ time + gluc, data = dfTest, repetition = ~time|id, method = "ML"))
logLik(lmm(weight ~ time + gluc*ff, data = dfTest, repetition = ~time|id, method = "ML"))
```

[1] -230.62

[1] -230.44

This is why, unless one knows what he/she is doing, it is not recommended to use likelihood ratio test to assess relevance of mean parameters in mixed models estimated with REML.

Appendix D Sum of squares in a linear mixed model

All mixed models implemented in LMMstar can be written as:

$$Y_{it} = X_{it}\beta + \varepsilon_{it} \text{ where } \varepsilon_i \sim \mathcal{N}(0, \Omega)$$

where Y denote the outcome repeatedly measured within each cluster i where t indexes the repetitions. X denotes the covariates, β the mean parameters, ε the residuals, and Ω the residual variance-covariance matrix. Ω must be positive definite so there must exist a square positive definite matrix $\Omega^{1/2}$ such that $\Omega^{1/2}\Omega^{1/2} = \Omega$. Therefore the previous model is equivalent to:

$$Y_{it}^* = X_{it}^*\beta + \varepsilon_{it}^* \text{ where } \varepsilon_i \sim \mathcal{N}(0, I_T)$$

where $Y_i^* = \Omega^{-1/2}Y_i$, $X_i^* = \Omega^{-1/2}X_i$, $\varepsilon_i^* = \Omega^{-1/2}\varepsilon_i$, and I_x is the identity matrix with x rows and columns. One can then introduce the projectors $H = X(X^\top\Omega^{-1}X)^{-1}X^\top\Omega^{-1}$ and $H^* = X^*(X^{*\top}X^*)^{-1}X^{*\top}$ onto the space spanned by X and X^* respectively. We can now define the "normalized" residual sum of squares as the squared sum of the normalized residuals:

$$\begin{aligned} SSE^* &= \varepsilon^{*\top}\varepsilon^* = Y^{*\top}(I_{nT} - H^*)Y^* \\ &= Y^\top\Omega^{-1}Y - Y^\top\Omega^{-1}X(X^\top\Omega^{-1}X)^{-1}X^\top\Omega^{-1}Y \\ &= Y^\top(I_{nT} - H^\top)\Omega^{-1}(I_{nT} - H)Y \end{aligned}$$

The previous to last line uses that: $(I_{nT} - H^\top)\Omega^{-1}(I_{nT} - H) = \Omega^{-1} - H^\top\Omega^{-1} - \Omega^{-1}H + H^\top\Omega^{-1}H = \Omega^{-1} - H^\top\Omega^{-1}$ as $H^\top\Omega^{-1}H = \Omega^{-1}HH = \Omega^{-1}H$ since H is a projector. Note that compared to the "traditional" SSE defined for linear regression and random effect models (e.g. see [Christensen \(2011\)](#) section 2.7), $SSE = \delta SSE^*$ where δ is the residual variance conditional on any random effects, i.e. SSE^* are the residual degrees of freedom. This is because the same definition for the sum of squares is used except that $\varepsilon_i \sim \mathcal{N}(0, \delta\Omega)$.

We can also define the "normalized" regression sum of squares:

$$\begin{aligned} SSR^* &= (X^*\beta)^\top X^*\beta = (H^*Y^*)^\top H^*Y^* = Y^{*\top}H^*Y^* = Y^{*\top}H^{*\top}H^*Y^* \\ &= Y^\top\Omega^{-1/2}\Omega^{-1/2}X(X^\top\Omega^{-1}X)^{-1}X^\top\Omega^{-1/2}\Omega^{-1/2}X(X^\top\Omega^{-1}X)^{-1}X^\top\Omega^{-1/2}\Omega^{-1/2}Y \\ &= ((X^\top\Omega^{-1}X)^{-1}X^\top\Omega^{-1}Y)^\top X^\top\Omega^{-1}X((X^\top\Omega^{-1}X)^{-1}X^\top\Omega^{-1}Y) \\ &= \hat{\beta}X^\top\Omega^{-1}X\hat{\beta} \end{aligned}$$

where $\hat{\beta} = (X^\top\Omega^{-1}X)^{-1}X^\top\Omega^{-1}Y$. Note that when using the expected information $SSR^* = \hat{\beta}\Sigma_{\hat{\beta}}^{-1}\hat{\beta}$, i.e. it is the F-statistics times the number of parameters. Again the "traditional" SSR defined for linear regression and random effect models is proportional to this normalized SSR: $SSR = \delta SSR^*$.

The proportion of explained variance of p parameters can thus be re-expressed as:

$$R^2 = \frac{SSR}{SSR + SSE} = \frac{SSR^*}{SSR^* + SSE^*} = \frac{Fp}{Fp + df}$$

where df denotes the residual degrees of freedom, typically $n - p$ in a univariate linear model fitted

with n observations.

⚠ In practice df is estimated using the Satterthwaite approximation of the degrees of freedom of the regression coefficient. This is only equivalent to the "SSR/SSE" formula in univariate linear regression.

Illustration for a univariate linear model:

Data without missing values:

```
df.aov <- gastricbypassL[!is.na(gastricbypassL$glucagon),]
```

Traditional anova decomposition:

```
e.lm <- lm(weight ~ visit + glucagonAUC, data = df.aov)
car::Anova(e.lm, type = "II")
```

Anova Table (Type II tests)

```
Response: weight
              Sum Sq Df F value Pr(>F)
visit          5837   3    5.94 0.0011 **
glucagonAUC     2133   1    6.51 0.0128 *
Residuals     23925  73
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Fit lmm:

```
e.lmm <- lmm(weight ~ visit + glucagonAUC, data = df.aov)
```

Residual sum of squares (SSE):

```
SSEstar <- crossprod(residuals(e.lmm, type = "normalized"))
c(SSEstar = SSEstar, SSE = SSEstar * sigma(e.lmm))
```

```
SSEstar    SSE
      73    23925
```

The normalized SSE can also be obtained using the `df.residual` method:

```
df.residual(e.lmm)
```

```
[1] 73
```

Regression sum of squares (SSR):

```
eBeta.lmm <- coef(e.lmm)
eVcov.lmm <- vcov(e.lmm, type.information = "expected")

SSRstar.glucagon <- eBeta.lmm[5] %*% solve(eVcov.lmm[5,5]) %*% eBeta.lmm[5]
SSRstar.time <- eBeta.lmm[2:4] %*% solve(eVcov.lmm[2:4,2:4]) %*% eBeta.lmm[2:4]
```

```
c(SSR.glucagon = SSRstar.glucagon * sigma(e.lmm),
  SSR.time = SSRstar.time * sigma(e.lmm),
  F.glucagon = SSRstar.glucagon,
  F.time = SSRstar.time/3)
```

```
SSR.glucagon    SSR.time    F.glucagon    F.time
      2132.629      5837.410        6.507        5.937
```

So the proportion of explained variance is:

```
R2.glucagon <- SSRstar.glucagon/(SSRstar.glucagon+SSEstar)
R2.glucagon
```

```
      [,1]
[1,] 0.081842
```

and the corresponding partial correlation is:

```
sign(coef(e.lmm)["glucagonAUC"])*sqrt(R2.glucagon)
```

```
      [,1]
[1,] -0.28608
```

which matches the output of `partialCor`:

```
summary(partialCor(e.lmm, R2 = TRUE))
```

Partial correlation

	estimate	se	df	lower	upper	p.value
visit2	-0.151	0.113	73	-0.377	0.074	0.18450
visit3	-0.013	0.117	73	-0.246	0.22	0.91230
visit4	-0.381	0.092	73	-0.565	-0.197	< 1e-04
glucagonAUC	-0.286	0.103	73	-0.491	-0.081	0.00695

Columns lower and upper contain 95% pointwise confidence intervals for each coefficient. Degrees of freedom were computed using a Satterthwaite approximation (column df).

Coefficient of determination (R2)

	estimate	se	df	lower	upper	p.value
visit	0.196	0.075	73	0.047	0.345	0.010548
glucagonAUC	0.082	0.059	73	-0.036	0.199	0.169016
global	0.29	0.075	73	0.14	0.44	0.000257

Columns lower and upper contain 95% pointwise confidence intervals for each coefficient. Degrees of freedom were computed using a Satterthwaite approximation (column df).

Appendix E Equivalence with other R packages

E.1 nlme package

The model class obtained with the `lmm` function overlaps the model class of the `lme` and `gls` functions from the `nlme` package.

```
library(nlme)
```

For instance, the compound symmetry is equivalent to `corCompSymm` correlation structure, or to a random intercept model (when the within subject correlation is positive):

```
eRI.lmm <- lmm(weight ~ visit*group, structure = "RE",
              data = gastricbypassL, repetition = ~visit|id)
eCS.gls <- gls(weight ~ visit*group, correlation = corCompSymm(form=~visit|id),
              data = gastricbypassL, na.action = na.omit)
eCS.lme <- lme(weight ~ visit*group, random = ~1|id,
              data = gastricbypassL, na.action = na.omit)
logLik(eRI.lmm)
logLik(eCS.lme)
logLik(eCS.gls)
```

```
[1] -236.21
'log Lik.' -236.21 (df=10)
'log Lik.' -236.21 (df=10)
```

The estimated random effect also match:

```
range(ranef(eRI.lmm)-ranef(eCS.lme))
```

```
[1] -1.7303e-08  2.6979e-08
```

Unstructured residual covariance matrix can also be obtained with `gls`:

```
eUN.gls <- gls(glucagonAUC ~ visit*group,
              correlation = corSymm(form=~as.numeric(visit)|id),
              weights = varIdent(form=~1|visit),
              data = gastricbypassL, na.action = na.omit)
logLik(eUN.gls)
logLik(eUN.lmm)
```

```
'log Lik.' -295.31 (df=18)
[1] -295.31
```

E.2 lme4 package

The model class obtained with the `lmm` function overlaps the model class of the `lmer` function from the `lme4` package.

```
library(lme4)
library(lmerTest)
```

For instance, the compound symmetry is equivalent to a random intercept model (when the within subject correlation is positive):

```
eRI.lmer <- lmer(weight ~ visit*group + (1|id),
               data = gastrichypassL)
logLik(eRI.lmer)
logLik(eRI.lmm)
```

```
'log Lik.' -236.21 (df=10)
[1] -236.21
```

The estimated random effects match:

```
range(ranef(eRI.lmm)-ranef(eRI.lmer)$id)
```

```
[1] -1.5513e-08  2.4171e-08
```

Nested random effects correspond to block unstructured:

```
eNRI.lmm <- lmm(weight ~ visit*group, structure = RE(~(1|id/baseline)),
               data = gastrichypassL, repetition = ~visit|id)
eNRI.lmer <- lmer(weight ~ visit*group + (1|id/baseline),
                data = gastrichypassL)
logLik(eNRI.lmer)
logLik(eNRI.lmm)
```

```
'log Lik.' -234.97 (df=11)
[1] -234.97
```

And the estimated random effects still match:

```
eRanefNRI.lmm <- ranef(eNRI.lmm, format = "wide")
eRanefNRI.lmer <- ranef(eNRI.lmer)
## id
range(eRanefNRI.lmm$estimate-eRanefNRI.lmer$id)
## baseline
range(c(eRanefNRI.lmm$estimate.FALSE,eRanefNRI.lmm$estimate.TRUE)-ranef(eNRI.lmer)$'baseline:
      id')
```

```
[1] -5.8317e-06  9.0913e-06
[1] -8.5850e-05  7.8971e-05
```

An unstructure residual covariance matrix can also be obtained using random slopes:

```
eUN.lmer <- lmer(glucagonAUC ~ visit*group + (0 + visit|id),
  data = gastrichbypassL,
  control = lmerControl(check.nobs.vs.nRE = "ignore"))
logLik(eUN.lmer)
logLik(eUN.lmm)
```

Warning message:

```
In checkConv(attr(opt, "derivs"), opt$par, ctrl = control$checkConv,  :
  Model failed to converge with max|grad| = 0.00203036 (tol = 0.002, component 1)
'log Lik.' -295.31 (df=19)
[1] -295.31
```

The uncertainty is quantified in a slightly different way, e.g.:

```
anova(eUN.lmm)
```

Multivariate Wald test

	F-statistic	df	p.value
mean: visit	5.803 (3,16.9)	0.00647	**
: group	3.926 (1,18.0)	0.06302	.
: visit:group	2.762 (3,17.3)	0.07332	.

is very similar but not identical to:

```
## only the last line is comparable
anova(eUN.lmer)
```

Type III Analysis of Variance Table with Satterthwaite's method

	Sum Sq	Mean Sq	NumDF	DenDF	F value	Pr(>F)
visit	1339	446	3	17.4	18.29	1.3e-05 ***
group	5	5	1	18.1	0.22	0.647
visit:group	203	68	3	17.4	2.77	0.073 .

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

It is also possible to fit cross-random effects such as:

```
data("Penicillin")
eCRI.lmer <- lmer(diameter ~ 1 + (1|plate) + (1|sample), Penicillin)
logLik(eCRI.lmer)
```

```
'log Lik.' -165.43 (df=4)
```

using lmm:

```

Penicillin$index <- paste(Penicillin$sample, Penicillin$plate, sep=".")
Penicillin$id <- 1

eCRI.lmm <- lmm(diameter ~ 1 + (1|plate) + (1|sample), data = Penicillin)
logLik(eCRI.lmm)

```

```
[1] -165.43
```

Despite being significantly slower, the loglikelihood and random effect still match:

```

range(ranef(eCRI.lmm)$estimate-rbind(ranef(eCRI.lmer)$plate, ranef(eCRI.lmer)$sample))

```

```
[1] -4.3812e-07  6.0172e-07
```

E.3 mmrm package

The package `mmrm` is an alternative implementation of mixed models specified via covariance structures:

```

library(mmrm)
e.mmrm <- mmrm(
  formula = FEV1 ~ RACE + SEX + ARMCD * AVISIT + us(AVISIT | USUBJID),
  data = fev_data
)

```

It leads nearly identical results compared to `lmm`:

```

e.lmm <- lmm(
  formula = FEV1 ~ RACE + SEX + ARMCD * AVISIT,
  repetition = ~ AVISIT | USUBJID, structure = "UN",
  data = fev_data, type.information = "expected"
)

```

Warning message:

```

In .lmmNormalizeData(as.data.frame(data)[unique(stats::na.omit(var.all))], :
  3 clusters have been removed.

```

```

logLik(e.mmrm) - logLik(e.lmm)
range(coef(e.mmrm) - coef(e.lmm))
range(vcov(e.mmrm) - vcov(e.lmm))

```

```
[1] -2.5413e-06
```

```
[1] -0.00018301  0.00016268
```

```
[1] -0.00039710  0.00020479
```

The main differences are:

- `mmrm` uses the expected information matrix to quantify uncertainty instead of the observed information matrix.
- `mmrm` implements the Kenward and Roger method for computing the degrees of freedom and not only the Satterthwaite approximation
- `mmrm` implements different covariance patterns
- `mmrm` is faster and probably more memory efficient
- `mmrm` has currently fewer post-processing methods (e.g. adjustment multiple comparisons when testing several model parameters). This being said, the latest version of the package (0.3.7) included several additional extractor of model feature so this may be improved in the future.

E.4 emmeans package

To illustrate a key difference between the `emmeans` package and the `effects.lmm` function we consider an informative and unbalanced group variable:

```
gastricbypassLB$group2 <- gastricbypassLB$weight1>150
```

Since `lmm`:

```
eCS.lmm_2 <- lmm(glucagonAUC ~ visit*group2, repetition =~visit|id, structure = "CS", data =
  gastricbypassLB)
logLik(eCS.lmm_2)
```

```
[1] -315.2
```

we will use the equivalent with the random effect specification:

```
eRI.lmer_2 <- lmer(glucagonAUC ~ visit*group2 + (1|id), data = gastricbypassLB)
logLik(eRI.lmer_2)
```

```
'log Lik.' -315.2 (df=10)
```

While the two models are equivalent, the average outcome output by `effects`:

```
effects(eCS.lmm_2, variable = NULL)
```

Average counterfactual outcome

	estimate	se	df	lower	upper
(t=1)	32.317	4.426	64.3	23.476	41.158
(t=2)	29.653	4.535	65.2	20.598	38.709
(t=3)	77.308	4.535	65.1	68.25	86.366
(t=4)	51.95	4.426	64.3	43.109	60.791

substantially differ from the one of `emmeans`:

```
library(emmeans)
emmeans(eRI.lmer_2, specs=~visit)
```

NOTE: Results may be misleading due to involvement in interactions

visit	emmean	SE	df	lower.CL	upper.CL
1	33.6	5.53	64.2	22.6	44.7
2	32.0	5.57	64.4	20.9	43.2
3	70.0	5.57	64.4	58.9	81.1
4	47.2	5.53	64.2	36.1	58.2

Results are averaged over the levels of: group2
 Degrees-of-freedom method: kenward-roger
 Confidence level used: 0.95

This is because when averaging over the level of a covariate, emmeans considers *balanced groups*. In the example, the groups are not balanced:

```
table(gastricbypassLB$group2)/NROW(gastricbypassLB)
```

```
FALSE TRUE
 0.8   0.2
```

Based on the group and timepoint specific means:

```
eCS.elmm_2 <- model.tables(effects(eCS.lmm_2, variable = "group2"))
eCS.elmm_2
```

	group2	visit	estimate	se	df	lower	upper	p.value
1	FALSE	1	31.430	4.9484	64.349	21.545	41.314	2.4688e-08
2	FALSE	2	28.067	5.0996	65.383	17.884	38.251	6.6737e-07
3	FALSE	3	82.173	5.1008	65.211	71.986	92.359	0.0000e+00
4	FALSE	4	55.126	4.9484	64.349	45.241	65.010	0.0000e+00
5	TRUE	1	35.864	9.8967	64.349	16.095	55.633	5.7374e-04
6	TRUE	2	35.997	9.8967	64.349	16.228	55.766	5.4953e-04
7	TRUE	3	57.848	9.8967	64.349	38.079	77.617	1.8339e-07
8	TRUE	4	39.246	9.8967	64.349	19.477	59.015	1.8651e-04

We illustrate the difference:

- emmeans:

```
0.5*eCS.elmm_2[eCS.elmm_2$group2==FALSE,"estimate"]+0.5*eCS.elmm_2[eCS.elmm_2$group2==TRUE,"estimate"]
```

```
[1] 33.647 32.032 70.010 47.186
```

- effects:

```
0.8*eCS.elmm_2[eCS.elmm_2$group2==FALSE,"estimate"]+0.2*eCS.elmm_2[eCS.elmm_2$group2==TRUE,"estimate"]
```

```
[1] 32.317 29.653 77.308 51.950
```

The "emmeans" approach gives equal "weight" to the expected value of both group:

```
mu.group1 <- as.double(coef(e.group)["(Intercept)"])
mu.group2 <- as.double(coef(e.group)["(Intercept)"] + coef(e.group)["group2TRUE"])
p.group1 <- 14/20 ; p.group2 <- 6/20
c(emmeans = (mu.group1+mu.group2)/2, predict = mu.group1 * p.group1 + mu.group2 * p.group2)
```

```
emmeans predict
4.450435 4.514352
```

E.5 effectsize package (R^2 or η^2)

Partial η^2 can be computed based on `lmer` using the `effectsize` package:

```
library(effectsize)
eta_squared(eCS.lmer)
cat("\n")
```

```
# Effect Size for ANOVA (Type III)
```

Parameter	Eta2 (partial)	95% CI
visit	0.64	[0.50, 1.00]
group	0.01	[0.00, 1.00]
visit:group	0.19	[0.03, 1.00]

```
- One-sided CIs: upper bound fixed at
```

and are approximately equal to what one can compute "manually":

```
eCS.Wald <- anova(eCS.lmm)$multivariate
eCS.Wald$df.num*eCS.Wald$statistic/(eCS.Wald$df.num*eCS.Wald$statistic+eCS.Wald$df.denom)
```

```
[1] 0.335374 0.033811 0.186290
```

The will not be true for heteroschedastic models:

```
eUN.Wald <- anova(eUN.lmm)$multivariate
eUN.Wald$df.num*eUN.Wald$statistic/(eUN.Wald$df.num*eUN.Wald$statistic+eUN.Wald$df.denom)
```

```
[1] 0.50787 0.17905 0.32380
```

compared to:

```
eta_squared(eUN.lmer)
cat("\n")
```

```
# Effect Size for ANOVA (Type III)
```

Parameter	Eta2 (partial)	95% CI
visit	0.76	[0.54, 1.00]
group	0.01	[0.00, 1.00]
visit:group	0.32	[0.00, 1.00]

```
- One-sided CIs: upper bound fixed at
```

But in that case both may be misleading as the proportion of explained variance is not clearly defined.

E.6 MuMIn package (R^2)

```
library(MuMIn)
r.squaredGLMM(eCS.lmer)
cat("\n")
```

```
      R2m      R2c
[1,] 0.51728 0.62222
```

To reproduce these R^2 , we extract from the random intercept model:

- the residual variance

```
sigmaW <- sigma(eCS.lmm)[1,1]-sigma(eCS.lmm)[1,2]
```

- the variance of the random effect

```
sigmaB <- sigma(eCS.lmm)[1,2]
```

- the variance of the fitted values:

```
sigma2_XB <- var(fitted(eCS.lmm))
```

and evaluate the ratios:

```
c(R2m = sigma2_XB/(sigmaW + sigmaB + sigma2_XB),
  R2c = (sigma2_XB + sigmaB)/(sigmaW + sigmaB + sigma2_XB))
```

```
      R2m      R2c
0.52549 0.62865
```

E.7 stats package (partial residuals)

The function `residuals.lm` can be used to extract partial residuals from `lm` objects. For instance:

```
gastricbypassW$group <- as.factor(as.numeric(gastricbypassW$id)%%2)
eIID.lm <- lm(weight4 ~ group + weight1, data = gastricbypassW)
pRes.lm <- residuals(eIID.lm, type = "partial")
head(pRes.lm)
```

```
      group weight1
1    7.19282   3.6648
2   -0.20504  31.7052
3    0.60631 -17.3352
4    6.44389  22.7052
5   -1.59403 -16.7352
6  -18.23382   8.4052
```

Those generally differ (by a constant) from the one provided by `residuals.lmm`:

```
eIID.lmm <- lmm(weight4 ~ group + weight1, data = gastricbypassW)
(residuals(eIID.lmm, type = "partial", variable = "group") - pRes.lm[, "group"])
(residuals(eIID.lmm, type = "partial", variable = "weight1") - pRes.lm[, "weight1"])
```

```

      1      2      3      4      5      6      7      8      9     10     11     12     13     14
2.0702 2.0702 2.0702 2.0702 2.0702 2.0702 2.0702 2.0702 2.0702 2.0702 2.0702 2.0702 2.0702 2.0702
      15     16     17     18     19     20
2.0702 2.0702 2.0702 2.0702 2.0702 2.0702
      1      2      3      4      5      6      7      8      9     10     11     12     13     14
106.22 106.22 106.22 106.22 106.22 106.22 106.22 106.22 106.22 106.22 106.22 106.22 106.22 106.22
      15     16     17     18     19     20
106.22 106.22 106.22 106.22 106.22 106.22
```

Indeed, `residuals.lm` centers the design matrix of the variable relative to which the partial residuals are computed:

```
coef(eIID.lm) ["group1"] * mean(gastricbypassW$group=="1")
coef(eIID.lm) ["weight1"] * mean(gastricbypassW$weight1)
```

```
group1
2.0702
weight1
106.22
```

For continuous variable with a linear effect, these residuals can be obtained by setting the `type` argument to `"partial-center"`:

```
(residuals(eIID.lmm, type = "partial-center", variable = "weight1") - pRes.lm[, "weight1"])
```

```

      1      2      3      4      5      6      7      8
1.7675e-13 6.7502e-14 -6.3949e-14 5.6843e-14 -3.9080e-14 8.1712e-14 -3.7303e-14 5.9508e-14
      9     10     11     12     13     14     15     16
-4.2633e-14 4.4409e-14 -2.9310e-14 5.5123e-14 -4.6185e-14 4.4409e-14 -4.2633e-14 4.6185e-14
      17     18     19     20
-3.9968e-14 5.3291e-14 -1.4211e-14 3.5527e-14
```

⚠ When evaluating the partial residuals relative to categorical variables, interactions, or non-linear terms, the output obtained with `partial-center` will not match the one of `residuals.lm`. Indeed `partial-center` will, when numeric, center the original variable whereas `residuals.lm` will center the column relative to the coefficient in the design matrix.